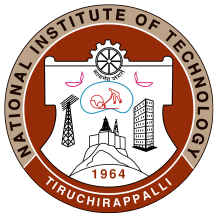
RGSW BASED UNIVERSALLY COMPOSABLE OBLIVIOUS TRANSFER

A thesis submitted in partial fulfillment of the requirements for the award of the degree of

B.Tech.
in
Computer Science and Engineering

By
Anurag Goyal (106119014)
Manmohan Prajapat (106119072)
Sobhagya Singh Dewal (106119117)



DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI-620015

MAY 2023

BONAFIDE CERTIFICATE

This is to certify that the project titled **RGSW BASED UNIVERSALLY COMPOSABLE OBLIVIOUS TRANSFER** is a bonafide record of the work done by

Anurag Goyal (106119014)

Manmohan Prajapat (106119072)

Sobhagya Singh Dewal (106119117)

in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI**, during the year 2022-23.

Dr. Kunwar Singh	**Dr. S. Mary Saira Bhanu**
Project Guide	Head of the Department

Project Viva-voce held on _____

Internal Examiner	**External Examiner**

ABSTRACT

Oblivious transfer (OT) is a cryptographic protocol that enables two parties to exchange information without revealing any information to each other. Many existing OT protocols base their security on classical hard problems such as prime factorization and the discrete logarithm problem. However, these problems can be solved by an ideal quantum computer but there exists no known classical or quantum algorithm which can break the LWE [1] and RLWE [2] hard problems. Gentry et al. [3] proposed a post-quantum secure assymetric homomorphic encryption scheme (GSW scheme) based on the LWE hard problem. We use the GSW scheme to construct its ring variant (RGSW) based on the RLWE hard problem. We further use the RGSW scheme along with the generic construction proposed in [4] to construct post-quantum secure 1-out-of-2 OT (OT_1^2) and 1-out-of-n OT (OT_1^n) protocols. Finally, we provide the proof of security in the universal composability (UC) model [5].

Keywords: Oblivious Transfer, Fully Homomorphic Encryption, GSW Scheme, RGSW Scheme, Post-quantum Cryptography, Universal Composability

ACKNOWLEDGEMENT

On the very outset of this report, we would like to express our sincere gratitude to our guide, **Dr. Kunwar Singh**, for his continuous guidance and constructive suggestions on the matter throughout the course of this project.

We are deeply grateful to **Dr. S. Mary Saira Bhanu**, Head of the Department of Computer Science and Engineering and all the members of the Departmental Project Evaluation Committee - **Dr. B. Nithya**, **Dr. A. Santhana Vijayan** and **Dr. Sitara K.** for their valuable inputs and feedback.

Finally, we would also like to thank the teaching and non-teaching staff of the Department of Computer Science and Engineering, our parents, friends and classmates who have been a constant pillar of support and encouragement throughout the project work.

Anurag Goyal (106119014)

Manmohan Prajapat (106119072)

Sobhagya Singh Dewal (106119117)

TABLE OF CONTENTS

Title **Page No.**

Abstract . . . . . **ii**

Acknowledgement . . . . . **iii**

Table of Contents . . . . . **iv**

List of Figures . . . . . **vii**

1 Introduction **1**

1.1 Overview 1

1.2 Post-Quantum Cryptography 1

1.3 Fully Homomorphic Encryption 2

1.3.1 Homomorphism in RSA 2

1.3.2 Previous FHE schemes based on LWE 3

1.3.3 An overview of RGSW scheme 3

1.4 Oblivious Transfer 3

1.4.1 Introduction 3

1.4.2 Public-Key OT 4

1.4.3 Homomorphic OT 4

1.4.4 Security 5

2 Preliminaries **6**

2.1 Basics of Lattices 6

2.1.1 What is a Lattice ? 6

2.1.2 What makes a Basis good ? 6

2.1.3 Learning with Errors 8

2.1.4 Ring Learning with Errors 9

2.2 The GSW Scheme 10

2.2.1 Introduction 10

2.2.2 Homomorphic Operations 10

2.3 Prerequisites 11

2.3.1 GSW Cryptosystem 12

3 The RGSW Scheme 16
3.1 Contributions 16
3.2 Converting GSW to RGSW 16
3.2.1 Polynomial Ring 16
3.3 Preliminaries of conversion 17
3.3.1 Bit Decomposition 17
3.3.2 Inverse Bit Decomposition 17
3.3.3 Flattening 17
3.3.4 Powers of Two 17
3.4 RGSW Cryptosystem 18
3.4.1 Setup 18
3.4.2 Secret Key Generation 18
3.4.3 Public Key Generation 18
3.4.4 Encryption 19
3.4.5 Decryption 19
3.4.6 Homomorphic Operations 20
3.5 Implementation 21
3.6 Results 25
3.6.1 Encryption and Decryption 25
3.6.2 Homomorphic Operations 26

4 Oblivious Transfer Protocol 28
4.1 Contributions 28
4.2 RGSW based 1-out-of-2 OT Protocol 28
4.2.1 Definition 28
4.2.2 Security Requirements 28
4.2.3 Proposed Protocol 29
4.2.4 Flow Diagram 30
4.2.5 Proof of Correctness 30
4.3 RGSW based 1-out-of-n OT Protocol 31
4.3.1 Definition 31
4.3.2 Security Requirements 31
4.3.3 Proposed Protocol 32
4.3.4 Flow Diagram 33
4.3.5 Proof of Correctness 33
4.4 Security Model 34
4.4.1 Universal Composability Framework 34

4.5 Implementation 34
4.6 Results 37

5 Conclusion 39
5.1 Work Done 39
5.2 Future Scope 39

Appendices 42

A Implementation of RGSW and OT protocol 43

B Simulation and Testing 44

List of Figures

3.1 Encryption/Decryption time VS $log_2 q$ at $n = 16$ 25
3.2 Encryption/Decryption time VS $log_2 n$ at $q = 1024$ 26
3.3 Add/MultConst time VS $log_2 q$ at $n = 16$ 26
3.4 Add/MultConst time VS $log_2 n$ at $q = 1024$ 27

4.1 Oblivious transfer time VS $log_2 q$ at $n = 16, N = 20$ 37
4.2 Oblivious transfer time VS $log_2 n$ at $q = 1024, N = 20$ 37
4.3 Oblivious transfer time VS N at $n = 16, q = 8192$ 38

Chapter 1

Introduction

1.1 Overview

We have constructed OT_1^2 and OT_1^n protocols. These protocols use a lattice based asymmetric cryptosystem which supports homomorphic operations. The protocol is secure in the UC model [5] and is post-quantum secure under the RLWE assumption [2]. First, we introduce the GSW scheme proposed by Gentry et al. [3] for achieving fully homomorphic encryption (FHE) based on the LWE hard problem. Further, we construct a ring variant of this scheme based on the RLWE assumption. This ring variant scheme will be referred to as **RGSW** in this document. Finally, we use the RGSW scheme to construct the OT protocols.

1.2 Post-Quantum Cryptography

Post-quantum cryptography is concerned with the security of cryptographic schemes against attacks by quantum computers. Quantum computers have the potential to break many of the currently used public-key encryption and digital signature schemes, which rely on the hardness of certain mathematical problems, such as integer factorization and discrete logarithm. Therefore, post-quantum cryptography aims to develop cryptographic schemes that are secure against attacks by quantum computers.

Lattice-based cryptography is one of the most promising candidates for post-quantum cryptography. Lattices are geometric structures that have many useful properties, such as being hard to approximate certain problems. Lattice-based cryptography relies on the hardness of certain problems in lattices, such as the shortest vector problem (SVP) and the closest vector problem (CVP), which are believed to be resistant to attacks by both classical and quantum computers.

1.3 Fully Homomorphic Encryption

Homomorphic Encryption refers to the property which allows the computation of a function over the plaintexts in encrypted form. Assume a public-key cryptosystem:

- $\text{KeyGen}\left(1^{\lambda}\right)$**:** Generates the public-private key pair (pk, sk) according to the security parameter λ.
- $\text{Enc}\left(pk, m\right)$**:** Encrypts a plaintext $m \in \mathcal{P}$ with pk and outputs $c \in \mathcal{C}$.
- $\text{Dec}\left(sk, c\right)$**:** Decrypts a ciphertext $c \in \mathcal{C}$ with sk and outputs $m \in \mathcal{P}$.

A homomorphic public-key cryptosystem has the additional property:

$$\text{Enc}\left(pk, f\left(m_1, \cdots, m_n\right)\right) = g\left(\text{Enc}\left(pk, m_1\right), \cdots, \text{Enc}\left(pk, m_n\right)\right)$$

where $x_i \in \mathcal{P} \; \forall \; 1 \leq i \leq n$
only if there exist two functions f and g such that:

$$f : \mathcal{P}^n \rightarrow \mathcal{P}, n \geq 1$$

$$g : \mathcal{C}^n \rightarrow \mathcal{C}, n \geq 1$$

A homomorphic encryption scheme is fully homomorphic (FHE) when the depth of the evaluation circuit represented by f and g has no upper bound. Similarly, there are Somewhat Homomorphic (SHE) and Leveled Homomorphic (LHE) schemes as well.

1.3.1 Homomorphism in RSA

RSA (Rivest-Shamir-Adleman) is a popular public key cryptosystem which can be described as:

Define a number $n = p \times q$ where p and q are large prime numbers. $\phi\left(n\right)$ denotes the Euler's totient value of n. Let $e : \text{GCD}\left(e, \phi\left(n\right)\right) = 1$. Now define $d = e^{-1} \mod \phi\left(n\right)$.

- $\text{KeyGen}\left(1^{\lambda}\right)$**:** Generates the public-private key pair $\left(\left(e, n\right), d\right)$ according to the security parameter λ.
- $\text{Enc}\left(e, n, m\right)$**:** Encrypts a plaintext $m \in \mathbb{Z}_n$ and outputs $c = m^e \mod n \in \mathbb{Z}_n$.
- $\text{Dec}\left(d, c\right)$**:** Decrypts a ciphertext $c \in \mathbb{Z}_n$ and outputs $m = c^d \mod n \in \mathbb{Z}_n$.

The RSA cryptosystem is fully homorphic for the functions f and g defined as:

$$f\left(m_1, \cdots, m_k\right) = \prod_{i=1}^{k} m_i \mod n$$

$$g\left(c_1, \cdots, c_k\right) = \prod_{i=1}^{k} c_i \mod n$$

where $m_i \in \mathbb{Z}_n, c_i = \mathrm{Enc}\left(e, n, m_i\right) \ \forall\, 1 \leq i \leq k.$

1.3.2 Previous FHE schemes based on LWE

Previous FHE schemes based on LWE, such as the one proposed by Gentry et al. [3], also uses dot products of n-dimensional vectors as ciphertexts and secret keys. However, homomorphic multiplication in these schemes requires a tensor product operation, which leads to a blowup in ciphertext size and requires relinearization, an expensive operation that involves large matrices and polynomial time complexity. The relinearization step is an ingenious insight and not intuitive or natural.

As seen in the GSW scheme, LWE-based schemes have the advantage of making bootstrapping optional and basing security on classical GapSVP, their performance is worse than RLWE-based schemes. RLWE-based schemes, however, cannot yet be considered practical. The per-gate complexity of evaluation and parameters depend on the multiplicative depth (L) of the circuit in both types of schemes, and bootstrapping remains the only known way to make these performance metrics independent of L. However, the overhead of bootstrapping is high, and it eliminates the space complexity advantages of LWE-based FHE schemes.

1.3.3 An overview of RGSW scheme

RGSW overcomes the difficulties faced by previous LWE-based FHE schemes and makes RLWE-based FHE schemes practically possible with polynomial space and time complexity. RGSW allows the same homomorphic operations as GSW without compromising on security. The major advantage of RGSW over GSW is the significant reduction in size for the private-public key pair and ciphertexts. Moreover, any arbitrary element of the ring can be used as plaintext in RGSW unlike other schemes where plaintext can be either 0 or 1.

1.4 Oblivious Transfer

1.4.1 Introduction

Oblivious transfer (OT) is a fundamental cryptographic protocol that enables two parties, the sender and the receiver, to exchange information without either party learning anything about the other's input. This protocol was first introduced by Rabin in 1981 [6] and has since become an essential tool in secure multiparty computation, where

multiple parties need to compute a function on their private data without revealing it to each other.

It involves a sender transferring one of the potentially many pieces of information to a receiver while remaining oblivious as to which piece has been transferred. In an OT protocol, the sender has n messages and the receiver has a choice b. The goal of the protocol is for the receiver to obtain the bth message from the sender without the sender learning b and for the receiver not to learn anything about the other $n - 1$ messages. The sender and receiver perform a series of interactions to achieve this goal, and the protocol is considered secure if it satisfies certain security properties.

1.4.2 Public-Key OT

One common way to achieve OT is through the use of public-key cryptography. In particular, the sender can encrypt each of their n messages using the receiver's public key, resulting in a set of n ciphertexts. The receiver can then use their private key to decrypt the bth ciphertext, obtaining the desired message without learning anything about the other messages.

This approach, known as public-key oblivious transfer (PKOT), can be extended to achieve more general secure multiparty computation. In particular, the parties can use PKOT as a building block to securely compute any function of their private inputs, without revealing any information about their inputs to each other.

1.4.3 Homomorphic OT

Another approach to achieving OT is through the use of homomorphic encryption. In particular, the sender can encrypt their messages using a homomorphic encryption scheme that supports addition and multiplication of ciphertexts. The receiver can then use their binary choice bit to compute a decryption circuit that either computes the first or second message, depending on the value of the bit. The sender can then evaluate this circuit on the encrypted messages and send the result to the receiver, who can use their secret key to decrypt the result.

This approach, known as homomorphic oblivious transfer (HOT), can be used to achieve more efficient secure computation than PKOT in certain scenarios. In particular, HOT can be used to compute functions that are difficult to express using a boolean circuit, such as arithmetic circuits.

1.4.4 Security

The security of OT protocols can be analyzed using various cryptographic tools, such as indistinguishability obfuscation, non-interactive zero-knowledge proofs, and fully homomorphic encryption. In particular, the security of PKOT can be reduced to the hardness of the Decisional Diffie-Hellman (DDH) assumption, while the security of HOT can be reduced to the hardness of the Learning With Errors (LWE) problem.

Despite the theoretical importance of OT, it has proven difficult to implement efficiently in practice. In particular, existing OT protocols are often computationally expensive and require large amounts of communication between the parties. Recent advances in lattice-based cryptography, however, have led to more efficient OT protocols that offer better performance than previous approaches.

Different variations of OT, such as OT_1^2, OT_1^n and OT_k^n, have been proposed in the literature. The universal composability (UC) [5] security model is considered for OT due to its use as a building block in other cryptographic primitives.

Chapter 2

Preliminaries

2.1 Basics of Lattices

2.1.1 What is a Lattice ?

A basis is a set of linearly independent vectors. A lattice is a set of all linear combinations with *integer* coefficients of the subgroup of any basis $\mathbb{R}^m$. A lattice $\mathcal{L}$ is a discrete subgroup of H generated by all the integer combinations of the vectors of some basis $\mathbf{B} \in \mathbb{R}^{n \times m}$:

$$\mathcal{L}(\mathbf{B}) = \sum_{i=1}^{n} \mathbb{Z}\mathbf{b_i} = \left\{ \sum_{i=1}^{n} z_i \mathbf{b_i} \mid z_i \in \mathbb{Z}, \mathbf{b_i} \in \mathbf{B} \right\}$$

Given m linearly independent vectors $\mathbf{b}_1, \mathbf{b}_2, \cdots, \mathbf{b}_n \in \mathbb{R}^m$ which form a basis $\mathbf{B}$, their span is defined as:

$$\mathrm{Span}(\mathbf{B}) = \left\{ \sum_{i=1}^{n} x_i \mathbf{b_i} \mid x_i \in \mathbb{R}, \mathbf{b_i} \in \mathbf{B} \right\}$$

The difference between $\mathcal{L}(\mathbf{B})$ and $\mathrm{Span}(\mathbf{B})$ is that $\mathcal{L}(\mathbf{B})$ consists of all linear combinations with *integer* coefficients, whereas, $\mathrm{Span}(\mathbf{B})$ consists of all linear combinations with *real* coefficients. Therefore,

$$\mathrm{Span}(\mathbf{B}) \supset \mathcal{L}(\mathbf{B})$$

2.1.2 What makes a Basis good ?

Any lattice has infinitely many bases, i.e, different linear combinations of different bases can generate the same lattice. Only the length of vectors and their integer coefficients differ. It is desirable to find a basis which has short vectors and the vectors as much orthogonal as possible. Having such a basis for a given lattice makes it easier to solve some hard problems based on lattices.

Basis Reduction

The purpose of basis reduction in lattices is to find a more efficient and simpler basis for a given lattice that can provide better exploitation of geometry and easier computation of the properties of the lattice. Shorter vectors imply faster computation of lattice reduction algorithms such as LLL and BKZ.

Gram-Schmidt Orthogonalization

There is an algorithm which can compute a set of orthogonal vectors corresponding to a given set of vectors. This algorithm is known as Gram-Schmidt orthogonalization. Let there be a set of vectors $\mathbf{b_1}, \mathbf{b_2}, \cdots, \mathbf{b_m} \in \mathbb{R}^n$, the corresponding set of orthogonal vectors $\overline{\mathbf{b}}_\mathbf{1}, \overline{\mathbf{b}}_\mathbf{2}, \cdots, \overline{\mathbf{b}}_\mathbf{m} \in \mathbb{R}^n$ is given by:

$$\overline{\mathbf{b}}_\mathbf{i} = \mathbf{b_i} - \sum_{j=1}^{i-1} \mu_{i,j} \overline{\mathbf{b}}_\mathbf{j} \text{ where } \mu_{i,j} = \frac{\langle \mathbf{b_i}, \overline{\mathbf{b}}_\mathbf{j} \rangle}{\langle \overline{\mathbf{b}}_\mathbf{j}, \overline{\mathbf{b}}_\mathbf{j} \rangle} \; \forall \, 1 \leq i \leq m$$

$\langle \mathbf{x}, \mathbf{y} \rangle$ is the dot product of two vectors $\mathbf{a}$ and $\mathbf{b}$.

The generated set $\overline{\mathbf{b}}_\mathbf{1}, \overline{\mathbf{b}}_\mathbf{1}, \cdots, \overline{\mathbf{b}}_\mathbf{m}$ is orthogonal, i.e, for each $i \neq j, \langle \overline{\mathbf{b}}_\mathbf{i}, \overline{\mathbf{b}}_\mathbf{j} \rangle = 0$. Also, $\text{Span}\left(\mathbf{b_1}, \mathbf{b_2}, \cdots, \mathbf{b_m}\right) = \text{Span}\left(\overline{\mathbf{b}}_\mathbf{1}, \overline{\mathbf{b}}_\mathbf{2}, \cdots, \overline{\mathbf{b}}_\mathbf{m}\right)$. An important thing to note here is that Gram-Schmidt orthogonalization does not necessarily produce a basis because the quantity $\mu_{i,j} \in \mathbb{R}$ has a *real* value, whereas, a basis matrix $\mathbf{B}$ should have *integer* values.

Lenstra-Lenstra-Lovász

To find a basis with short vectors, an adaptation of Gram-Schmidt orthogonalization is used where $\lfloor \mu_{i,j} \rceil$ is used instead of $\mu_{i,j}$, where $\lfloor \cdot \rceil$ means rounding to the closest integer. This adaptation is employed in the LLL basis reduction algorithm which was invented by Lenstra, Lenstra and Lovász in 1982 [7]. It takes a basis $\mathbf{B} = (\mathbf{b_1}, \mathbf{b_2}, \cdots, \mathbf{b_m})$ as input and outputs a LLL-reduced basis. The algorithm works as follows:

Step 1 for $i = 1$ to n:
    for $k = i - 1$ to 1:
        $\mathbf{b_i} \leftarrow \lfloor \mu_{k,i} \rceil \mathbf{b_k}$

Step 2 for $i = 1$ to $n - 1$:
    if $\|\overline{\mathbf{b}}_{i+1} + \mu_{i,i+1} \overline{\mathbf{b}}_i\|^2 < \frac{3}{4} \|\overline{\mathbf{b}}_i\|^2$:
        $\text{swap}\left(\overline{\mathbf{b}}_i, \overline{\mathbf{b}}_{i+1}\right)$
        goto step 1

The LLL algorithm gives an approximate solution to the SVP problem in an n-dimensional lattice in polynomial time. Several existing encryption schemes can be broken by constructing a lattice and then finding the approximate shortest vector using LLL basis reduction. That is why lattices are important in mathematics and cryptography, and the Shortest Vector Problem (SVP) is the most studied problem involving lattices. SVP involves finding the shortest nonzero vector in a given lattice, usually defined by the Euclidean norm. Another variant of SVP involves finding the length of the shortest non-zero vector without finding the vector itself. SVP is related to problems in number theory and integer linear programming and is of interest in cryptography due to its average hardness.

2.1.3 Learning with Errors

The Learning with Errors (LWE) problem was proposed by Oded Regev in 2005 [1] as a foundation for lattice-based cryptosystems with smaller key and ciphertext sizes. The LWE problem involves recovering a secret key from a set of noisy linear equations, which is significantly harder than solving non-noisy linear equations using Gaussian elimination.

Consider the system of linear equations

$$\mathbf{A} \cdot \mathbf{s} = \mathbf{b} \bmod q$$

Given $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{b} \in \mathbb{Z}_q^n$, such a system can be solved for $\mathbf{s}$ in polynomial time by Gaussian elimination. However, the above problem becomes hard to solve if the equality is removed by adding some small noise $\mathbf{e}$, i.e, the system of equations

$$\mathbf{A} \cdot \mathbf{s} + \mathbf{e} = \mathbf{b} \bmod q$$

Also written as

$$\mathbf{A} \cdot \mathbf{s} \approx \mathbf{b} \bmod q$$

Where $\mathbf{e} \in \mathbb{Z}_q^n$ is a random small error vector sampled from a distribution χ.

The search LWE problem is to find $\mathbf{s} \in \mathbb{Z}_q^m$ in polynomial time such that $\mathbf{A} \cdot \mathbf{s} + \mathbf{e} = \mathbf{b} \bmod q$ given only $\mathbf{A}$ and $\mathbf{b}$. Subsequently, the decisional LWE problem is to distinguish between $(\mathbf{A}, \mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} \bmod q)$ and a uniformly random distribution.

An example of a simple LWE based public-key cryptosystem is as follows:

- **Key generation:** The secret key is a random element $\mathbf{s} \in \mathbb{Z}^m$. The public key is $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ chosen uniformly at random where $n = \lceil log_2 q \rceil$ (number of bits in q).

- **Encryption:** We can encrypt a message $m \in \mathbb{Z}_q$. Let $\mathbf{m} \in \{0,1\}^n$ be a bit-vector containing the bits of m and choose $\mathbf{e} \sim \chi^n$ uniformly at random. The ciphertext is computed as $\mathbf{b} = \mathbf{A} \cdot \mathbf{s} + \mathbf{e} + \lfloor q/2 \rceil \cdot \mathbf{m} \bmod q$.

- **Decryption:** Compute $\mathbf{z} = \mathbf{b} - \mathbf{A} \cdot \mathbf{s} \bmod q = \lfloor q/2 \rceil \cdot m + \mathbf{e} \bmod q$. Retrieve the bits of m by rounding each element of $\mathbf{z}$ to $q/2$ or 0 (whichever is closest), and interpreting 0 as 0 and $q/2$ as 1.

LWE as well as its algebraic (ring) variant (RLWE), are believed to be hard problems for classical and quantum computers alike. Therefore, many post-quantum key exchange and signature schemes rely on these problems as a basis for their security. In fact, three out of the six finalists in NIST's PQC process use these problems. This suggests that LWE and RLWE are promising candidates for achieving security against quantum computers.

2.1.4 Ring Learning with Errors

Ring learning with errors (RLWE) [2] is a variation of LWE that offers benefits in terms of key sizes and computational efficiency. We use a specific ring $R = \mathbb{Z}[x]/\Phi_{2n}(x)$ where $\Phi_{2n}(x) = x^n + 1$ is the nth cyclotomic polynomial and n is a power of 2. We will define $R_q = R/qR$, i.e, the coefficients of an element in R_q are in $\mathbb{Z}_q$. Therefore,

$$\sum_{i=0}^{n-1} a_i \cdot x^i \in R_q \; \forall \; a_i \in \mathbb{Z}_q$$

For integers k, $q \geq 2$, power of two n, and an error distribution χ for short elements in R, we are given $\mathbf{a} = (a_1, a_2, \cdots, a_k) \in R_q^k$ and $\mathbf{b} = (b_1, b_2, \cdots, b_k) \in R_q^k$ where $b_i = a_i \cdot s + e_i \bmod qR$ for $s \in R_q$ and $e_i \sim \chi$. The goal of the RLWE problem is to find s.

The following RLWE based public-key cryptosystem is an analogue of the aforementioned LWE equivalent:

- **Key generation:** The secret key is a random element $s \in R_q$ and the public key is a random element $a \in R_q$.

- **Encryption:** We can encrypt any plaintext $m \in R_{\{0,1\}}$. The ciphertext is given by $b = a \cdot s + e + \lfloor q/2 \rceil \cdot m \bmod qR$ where $e \sim \chi$.

- **Decryption:** Compute $z = b - a \cdot s \bmod qR = \lfloor q/2 \rceil \cdot m + e \bmod qR$. Round each coefficient of z to $q/2$ or 0 (whichever is closest), and 0 is interpreted as 0 and $q/2$ as 1.

RLWE is defined in terms of ideal lattices, which have additional algebraic structures that allow for further reduction in public key size while retaining almost identical provable security. The problem originated from LWE and is based on the K-SVP and K-SIVP problems, and is believed to be hard due to the extra structure in the ideal lattices, which cannot be exploited to reduce the runtime for solving them.

2.2 The GSW Scheme

2.2.1 Introduction

The Gentry-Sahai-Waters (GSW) [3] scheme is a fully homomorphic encryption (FHE) scheme that was introduced in 2013. It is based on the learning with errors (LWE) problem and is one of the most efficient FHE schemes known to date. The GSW scheme enables arbitrary computations on encrypted data without the need for decryption, making it a powerful tool for privacy-preserving computation. The security of the GSW scheme relies on the hardness of the LWE problem, which is believed to be resistant to attacks by both classical and quantum computers.

This scheme overcomes the difficulties faced by previous LWE-based FHE schemes and makes practical FHE possible with polynomial time and efficient time complexity. The scheme is based on a fundamentally new technique that significantly reduces the space complexity from quasi-cubic to quasi-quadratic, which is a significant issue for LWE-based FHE schemes in practice. The scheme retains the advantages of other LWE-based FHE schemes such as optional bootstrapping, basing security on LWE for quasi-polynomial factors, eliminating modulus switching, and basing security directly on the hardness of classical GapSVP.

2.2.2 Homomorphic Operations

Let $\mathbf{C} \in \mathbb{Z}_q^{m \times n}$ be the encryption of the plaintext $z \in \mathbb{Z}_q$ under an LWE encryption scheme and $\mathbf{v} \in \mathbb{Z}_q^n$ be the secret key. The basic relation is:

$$\mathbf{C} \cdot \mathbf{v} = z \cdot \mathbf{v} + \mathbf{e}$$

The following homomorphic operations can be performed on ciphertexts $\mathbf{C}_1$ and $\mathbf{C}_2$.

Ciphertext Addition

$$\mathbf{C}_+ = \mathbf{C}_1 + \mathbf{C}_2$$
$$\mathbf{C}_+ \cdot v = (z_1 + z_2) \cdot \mathbf{v} + (\mathbf{e}_1 + \mathbf{e}_2) = z_+ \cdot \mathbf{v} + small$$

Ciphertext Multiplication

$$\mathbf{C}_\times = \mathbf{C}_1 \times \mathbf{C}_2$$
$$\mathbf{C}_\times \cdot \mathbf{v} = (z_1 z_2) \cdot \mathbf{v} + (z_2 \cdot \mathbf{e}_1 + C_1 \cdot \mathbf{e}_2) = z_\times \cdot \mathbf{v} + small$$

Ciphertext Scalar Multiplication

$$\mathbf{C}_\alpha = \alpha \cdot \mathbf{C}$$
$$\mathbf{C}_\alpha \cdot \mathbf{v} = (\alpha z) \cdot \mathbf{v} + (\alpha \cdot \mathbf{e}) = z_\alpha + small$$

These errors amplify with each homomorphic operation but the GSW scheme makes use of operations on bit representations of vectors for achieving error control in resulting ciphertexts.

2.3 Prerequisites

Bit representation of vectors show some interesting properties. Define a vector

$$\mathbf{a} = (a_1, a_2, \cdots, a_k) \in \mathbb{Z}_q^k$$

Let l be the number of bits in q, i.e, $l = \lceil log_2 q \rceil$ and $N = (n+1) \cdot l$.

Bit Decomposition

Bit Decomposition expands a vector by replacing each element by a vector of its own bits.

$$\mathbf{a}' = \text{BitDecomp}_{GSW}(\mathbf{a}) = (a_{1,0}, \cdots, a_{1,l-1}, \cdots, a_{k,0}, \cdots, a_{k,l-1}) \in \{0,1\}^{lk}$$

where $a_{i,j}$ denotes the jth least significant bit (LSB) in a_i.

Inverse Bit Decomposition

As the name suggests, Inverse Bit Decomposition transforms any vector of dimension lk to a vector of dimension k. Note that $\mathbf{a}'$ need not be a $0/1$ vector.

$$\mathbf{a} = \text{BitDecompInv}_{GSW}(\mathbf{a}') = \left(\sum_{i=0}^{l-1} 2^i a_{1,i}, \cdots, \sum_{i=0}^{l-1} 2^i a_{k,i} \right) \in \mathbb{Z}_q^k$$

Flattening

By flattening, we transform each element of a vector to $0/1$ without any change to the vector's dimension.

$$\text{Flatten}_{GSW}\left(\mathbf{a}'\right) = \text{BitDecomp}_{GSW}\left(\text{BitDecompInv}_{GSW}\left(\mathbf{a}'\right)\right) \in \{0,1\}^{lk}$$

Powers of Two

This is similar to BitDecomp with the difference that each vector is multiplied by powers of two instead of being decomposed to bits.

$$\text{PowersOf2}_{GSW}\left(\mathbf{a}\right) = \left(a_1, 2a_1, \cdots, 2^{l-1}a_1, \cdots, a_k, 2a_k, \cdots, 2^{l-1}a_k\right) \in \mathbb{Z}_q^{lk}$$

For a matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times n}$, all the above functions operate on individual rows of A. Some useful properties are:

1. $\langle \text{BitDecomp}_{GSW}\left(\mathbf{a}\right), \text{PowersOf2}_{GSW}\left(\mathbf{b}\right)\rangle = \langle \mathbf{a}, \mathbf{b}\rangle$
2. $\langle \mathbf{a}, \text{PowersOf2}_{GSW}\left(\mathbf{b}\right)\rangle = \langle \text{BitDecompInv}_{GSW}\left(\mathbf{a}\right), \mathbf{b}\rangle$
3. $\langle \text{BitDecompInv}_{GSW}\left(\mathbf{a}\right), \mathbf{b}\rangle = \langle \text{Flatten}_{GSW}\left(\mathbf{a}\right), \text{PowersOf2}_{GSW}\left(\mathbf{b}\right)\rangle$
4. $\text{Flatten}_{GSW}\left(\mathbf{C}\right) \cdot \mathbf{v} = \mathbf{C} \cdot \mathbf{v}$

The last property gives an insight into the noise control strategy of the GSW scheme since the product $\mathbf{C} \cdot \mathbf{v}$ is unaffected if $\mathbf{C}$ is replaced by $\text{Flatten}_{GSW}\left(\mathbf{C}\right)$ and the latter has only $0/1$ for elements.

2.3.1 GSW Cryptosystem

Functionalities of the GSW public-key cryptosystem are described in this section.

Setup

$\text{Setup}_{GSW}\left(1^{\lambda}\right)$: Generates the modulus $q = 2^k, k = k(\lambda)$, a lattice dimension parameter $n = n(\lambda)$ according to the security parameter λ. $N = (n+1) \cdot l$ where l is the number of bits in q. Naturally, $l = k + 1$.

$\mathcal{I}_N \in \{0,1\}^{N \times N}$ denotes the identity matrix of order N and $\chi_{\mu,\sigma}$ denotes a gaussian distribution which has mean μ and standard deviation σ.

Secret Key Generation

$\mathrm{SKGen}_{GSW}(q, n)$: takes q and n as inputs and outputs vectors $\mathbf{t}$ and $\mathbf{v}$.

Choose a vector $\mathbf{t} = (t_1, t_2, \cdots, t_n) \in \mathbb{Z}_q^n$ unirformly at random. Define

$$\mathbf{s} = (1, -t_1, -t_2, \cdots, -t_n) \in \mathbb{Z}_q^{n+1}$$

The secret key is

$$\mathbf{v} = \mathrm{PowersOf2}_{GSW}\left(\mathbf{s}\right)$$

Public Key Generation

$\mathrm{PKGen}_{GSW}(\mathbf{t})$: outputs a matrix $\mathbf{A} \in \mathbb{Z}_q^{m \times (n+1)}$.

Choose a matrix $\mathbf{B} \in \mathbb{Z}_q^{m \times n}$ uniformly at random and an error vector $\mathbf{e} \sim \chi^m \in \mathbb{Z}_q^m$.

$$\mathbf{b} = \mathbf{B} \cdot \mathbf{t} + \mathbf{e} \in \mathbb{Z}_q^m$$
$$\mathbf{A} = \left[\begin{array}{cc} \mathbf{b} & \mathbf{B} \end{array} \right] \in \mathbb{Z}_q^{m \times (n+1)}$$

The matrix $\mathbf{A}$ is the public key. Note that $\mathbf{A} \cdot \mathbf{s} = \mathbf{e}$.

Encryption

$\mathrm{Enc}_{GSW}(\mathbf{A}, z)$: takes $z \in \mathbb{Z}_q$ as input and outputs a matrix $\mathbf{C} \in \mathbb{Z}_q^{N \times N}$.

Choose a plaintext $z \in \mathbb{Z}_q$ and a uniformly random matrix $\mathbf{R} \in \{0, 1\}^{N \times m}$. The ciphertext is given by

$$\mathbf{C} = \mathrm{Flatten}_{GSW}\left(z \cdot \mathcal{I}_N + \mathrm{BitDecomp}_{GSW}(\mathbf{R} \cdot \mathbf{A})\right) \in \mathbb{Z}_q^{N \times N}$$

Decryption

$\mathrm{Dec}_{GSW}(\mathbf{v}, \mathbf{C})$: takes the ciphertext matrix $\mathbf{C}$ as input and outputs the decrypted plaintext $z \in \mathbb{Z}_q$.

Note that $q = 2^k$. According to [8], let the vector $\mathbf{g} = \left(1, 2, \cdots, 2^{k-1}\right) \in \mathbb{Z}_q^k$ be the geometric vector. Define a lattice

$$\mathbf{S}_k = \begin{bmatrix} 2 & 0 & \cdots & 0 & 0 \\ -1 & 2 & \cdots & 0 & 0 \\ 0 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & 2 & 0 \\ 0 & 0 & \cdots & -1 & 2 \end{bmatrix} \in \mathbb{Z}^{k \times k}$$

Observe that,

$$\mathbf{S}_k \cdot \mathbf{g} = \{0\}^k \bmod q \subset \mathbb{Z}_q^k$$
$$\det(\mathbf{S}_k) = 2^k = q$$

This implies that all the basis vectors are short. The first k elements of $\mathbf{v}$ are $1, 2, \cdots, 2^{k-1}$, therefore the first k elements of $\mathbf{C} \cdot \mathbf{v} = z \cdot \mathbf{v} + \mathbf{e}$ are given by

$$\mathbf{b} = z \cdot \mathbf{g} + \mathbf{e} = \left(z + e_0, 2z + e_1, \cdots, 2^{k-1}z + e_{k-1}\right) \in \mathbb{Z}_q^k$$

where $\mathbf{e}$ is a small error vector.

Let $z_0, z_1, \cdots, z_{k-1} \in \{0, 1\}$ be the bits of the message $z \in \mathbb{Z}_q$ such that

$$\sum_{i=0}^{k-1} 2^i z_j = z \in \mathbb{Z}_q$$

Let the elements of $\mathbf{b}$ be denoted by

$$\mathbf{b} = (b_0, b_1, \cdots, b_{k-1}) = \left(z + e_0, 2z + e_1, \cdots, 2^{k-1}z + e_{k-1}\right)$$

Compute z_0 by checking whether $b_{k-1} = 2^{k-1}z + e_{k-1} = \left(\frac{q}{2}\right) z_0 + e_{k-1} \bmod q$ is closer to 0 or $\frac{q}{2}$. More precisely,

$$z_0 = \left\langle b_{k-1} \notin \left[-\frac{q}{4}, \frac{q}{4}\right) \bmod q \right\rangle$$

where $\langle x \rangle = \begin{cases} 1 & , x \text{ is true} \\ 0 & , x \text{ is false} \end{cases}$

Subsequently, z_1 can be recovered from $b_{k-2} = 2^{k-2}z + e_{k-2} = 2^{k-2}(2z_1 + z_0) + e_{k-2} \bmod q$ by computing

$$z_1 = \left\langle b_{k-2} - 2^{k-2}z_0 \notin \left[-\frac{q}{4}, \frac{q}{4}\right) \bmod q \right\rangle$$

Similarly, all $z_i, 0 \leq i < k$ can be recovered. The general decryption algorithm is

initialize $z \leftarrow 0$
for $i = k-1, \cdots, 1, 0$:
$z \leftarrow z + 2^{k-1-i} \cdot \left\langle b_i - 2^i \cdot z \notin \left[-\frac{q}{4}, \frac{q}{4}\right) \bmod q \right\rangle$.

Output z as the decrypted plaintext.

Homomorphic Operations

For the purpose of constructing an OT protocol, we require only ciphertext addition and ciphertext scalar mutiplication. Therefore, we refrain from discussing ciphertext multiplication.

Ciphertext Addition:
$\mathrm{Add}_{GSW}\left(\mathbf{C}_1, \mathbf{C}_2\right)$: returns the noise-controlled sum of $\mathbf{C}_1$ and $\mathbf{C}_2$.

$$\mathbf{C}_+ = \mathrm{Flatten}_{GSW}\left(\mathbf{C}_1 + \mathbf{C}_2\right) \in \mathbb{Z}_q^{N \times N}$$

The correctness of this operation is immediate.

Ciphertext Scalar Multiplication:
$\mathrm{MultConst}_{GSW}\left(\alpha, \mathbf{C}\right)$: returns the noise-controlled scalar product of $\mathbf{C}$ and α.

$$\mathbf{C}_\alpha = \mathrm{Flatten}_{GSW}\left(\mathrm{Flatten}_{GSW}\left(\alpha \cdot \mathcal{I}_N\right) \cdot \mathbf{C}\right) \in \mathbb{Z}_q^{N \times N}$$

The error increases by a factor of at most N.

Chapter 3

The RGSW Scheme

3.1 Contributions

We propose a new public-key cryptographic scheme called Ring-GSW or RGSW, which is based on the GSW scheme. RGSW scheme is a more efficient version of the GSW scheme and does not compromise on its post-quantum security. We have made contributions towards developing this scheme by addressing the limitations of the existing GSW scheme which suffers from very large key and ciphertext sizes.

The proposed scheme incorporates a *ring* structure that enables a significant reduction in the size of keys and ciphertexts. To ensure the post-quantum security of the scheme, we have constructed a modified version of the GSW homomorphic encryption scheme that utilizes the RLWE assumption.

3.2 Converting GSW to RGSW

In order to understand the RGSW scheme, some preliminary concepts and conversion techniques need to be introduced. This includes understanding basic properties of polynomial rings. It also involves functionalities which will be used in RGSW, such as $\mathrm{BitDecomp}$, $\mathrm{Flatten}$ for RGSW. By grasping these concepts and techniques, it becomes possible to understand the RGSW scheme and its efficient and practical implementation of RLWE based FHE.

3.2.1 Polynomial Ring

In RGSW we use a specific polynomial ring $R = Z[x]/\Phi_{2n}(x)$ where $\Phi_{2n}(x)$ is the nth cyclotomic polynomial defined as $\Phi_{2n}(x) = x^n + 1$ with n as a power of 2. Define $R_q = R/qR$, i.e., all coefficients of a polynomial in R_q are under modulo q. An element of R_q looks like $u = \sum_{i=0}^{n-1} u_i \cdot x^i, u_i \in \mathbb{Z}_q$.

3.3 Preliminaries of conversion

Define $q = 2^k$ and a polynomial ring $R_q = R/qR$ where $R = \mathbb{Z}[x]/\Phi_{2n}(x)$ and $\Phi_{2n}(x) = x^n + 1$. Choose an element $a \in R_q$ for which, the RGSW variants are described below.

3.3.1 Bit Decomposition

Bit Decomposition transforms a polynomial to a vector. An element of this vector has only $0/1$ for coefficients.

$$\mathbf{a} = \mathrm{BitDecomp}_{RGSW}(a) = \left(\sum_{i=0}^{n-1} a_{i,0}x^i, \cdots, \sum_{i=0}^{n-1} a_{i,k-1}x^i\right) \in R^k_{\{0,1\}}$$

$a_{i,j} \in \{0, 1\}$ denotes the jth least significant bit (LSB) in a_i.

3.3.2 Inverse Bit Decomposition

Inverse Bit Decomposition transforms any vector of dimension k to a polynomial in R_q. Note that $\mathbf{a}'$ need not be a $0/1$ vector.

$$a = \mathrm{BitDecompInv}_{RGSW}(\mathbf{a}) = \sum_{i=0}^{n-1}\left(\sum_{j=0}^{k-1} 2^j a_{i,j}\right)x^i \in R_q$$

Here, $a_{i,j} \in \mathbb{Z}_q$ need not be only $0/1$.

3.3.3 Flattening

Flattening is a mathematical operation that transforms the elements of a vector into polynomials in R_q, where each coefficient of the polynomial is either 0 or 1, without changing the dimension of the vector.

$$\mathrm{Flatten}_{RGSW}(\mathbf{a}) = \mathrm{BitDecomp}_{RGSW}(\mathrm{BitDecompInv}_{RGSW}(\mathbf{a})) \in \{0,1\}^{lk}$$

3.3.4 Powers of Two

This is similar to $\mathrm{BitDecomp}$ with the difference that each element is a multiplied by a power of two instead of being a $0/1$ polynomial.

$$\mathrm{PowersOf2}_{RGSW}(a) = \left(a, 2a, \cdots, 2^{k-1}a\right) \in R^k_q$$

For a matrix $\mathbf{A} \in R^{m\times n}_q$, all the above functions operate on individual rows of A. Some useful properties are:

1. $\langle \text{BitDecomp}_{RGSW}(a), \text{PowersOf2}_{RGSW}(b) \rangle = a \cdot b$
2. $\langle a, \text{PowersOf2}_{RGSW}(b) \rangle = \langle \text{BitDecompInv}_{RGSW}(\mathbf{a}), b \rangle$
3. $\langle \text{BitDecompInv}_{RGSW}(\mathbf{a}), b \rangle = \langle \text{Flatten}_{RGSW}(\mathbf{a}), \text{PowersOf2}_{RGSW}(b) \rangle$
4. $\text{Flatten}_{RGSW}(\mathbf{C}) \cdot \mathbf{v} = \mathbf{C} \cdot \mathbf{v}$

3.4 RGSW Cryptosystem

Functionalities of the RGSW public-key cryptosystem are described in this section.

3.4.1 Setup

$\text{Setup}_{RGSW}\left(1^{\lambda}\right)$: Generates the modulus $q = 2^k, k = k(\lambda)$, a lattice dimension parameter $n = n(\lambda)$ which is a power of 2 according to the security parameter λ.
Define a polynomial ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$ and $N = 2k$.

$\mathcal{I}_N \in \{0,1\}^{N \times N}$ denotes the identity matrix of order N and $\chi_{\mu,\sigma}$ denotes a gaussian distribution which has mean μ and standard deviation σ.

3.4.2 Secret Key Generation

$\text{SKGen}_{RGSW}(q, n)$: takes q and n as inputs and outputs vectors t and $\mathbf{v}$.

Choose $t \in R_a$ uniformly at random and define

$$\mathbf{s} = (1, -t) \in R_q^2$$

The secret key is

$$\mathbf{v} = \text{PowersOf2}_{RGSW}(\mathbf{s})$$
$$\mathbf{v} = \left(1, 2, \cdots, 2^{k-1}, -t, -2t, \cdots, -2^{k-1}t\right) \in R_q^N$$

3.4.3 Public Key Generation

$\text{PKGen}_{RGSW}(t)$: outputs a matrix $\mathbf{A} \in R_q^{1\times 2}$.

Choose $b_1 \in R_q$ uniformly at random and an error polynomial $e \sim \chi \in R_q$.

$$b_2 = b_1 \cdot\ t + e \in R_q$$
$$\mathbf{A} = \begin{bmatrix} b_2 & b_1 \end{bmatrix} \in R_q^{1\times 2}$$

The matrix $\mathbf{A}$ is the public key. Note that $\mathbf{A} \cdot \mathbf{s} = e$.

3.4.4 Encryption

$\mathrm{Enc}_{RGSW}(\mathbf{A}, z)$: takes $z \in R_q$ as input and outputs a matrix $\mathbf{C} \in R_q^{N \times N}$.

Choose a plaintext $z \in R_q$ and a uniformly random vector $\mathbf{r} \in R_q^N$ such that the coefficients of any member polynomial are $0/1$. In other words,

$$\mathbf{r} = \left(\sum_{i=0}^{n-1} y_{0,i} \cdot x^i, \cdots, \sum_{i=0}^{n-1} y_{N-1,i} \cdot x^i \right) \in R_q^N$$

where $y_{i,j} \in \{0, 1\} \; \forall \; 0 \leq i < N, 0 \leq j < k$

The ciphertext is given by

$$\mathbf{C} = \mathrm{Flatten}_{RGSW} \left(z \cdot \mathcal{I}_N + \mathrm{BitDecomp}_{RGSW}(\mathbf{r} \cdot \mathbf{A}) \right) \in R_q^{N \times N}$$

3.4.5 Decryption

$\mathrm{Dec}_{RGSW}(\mathbf{v}, \mathbf{C})$: takes the ciphertext matrix $\mathbf{C}$ as input and outputs the decrypted plaintext $z \in R_q$.

Note that $q = 2^k$. According to [8], let the vector $\mathbf{g} = \left(1, 2, \cdots, 2^{k-1}\right) \in R_q^k$ be the geometric vector. Define a lattice

$$\mathbf{S}_k = \begin{bmatrix} 2 & 0 & \cdots & 0 & 0 \\ -1 & 2 & \cdots & 0 & 0 \\ 0 & -1 & \cdots & 0 & 0 \\ \vdots & \vdots & \cdots & \vdots & \vdots \\ 0 & 0 & \cdots & 2 & 0 \\ 0 & 0 & \cdots & -1 & 2 \end{bmatrix} \in \mathbb{Z}^{k \times k}$$

Observe that,

$$\mathbf{S}_k \cdot \mathbf{g} = \{0\}^k \bmod q \subset \mathbb{Z}_q^k$$
$$\det\left(\mathbf{S}_k\right) = 2^k = q$$

This implies that all the basis vectors are short. The first k elements of $\mathbf{v}$ are $1, 2, \cdots, 2^{k-1}$, therefore the first k elements of $\mathbf{C} \cdot \mathbf{v} = z \cdot \mathbf{v} + \mathbf{e}$ are given by

$$\mathbf{b} = z \cdot \mathbf{g} + e = \left(z + e_0, 2z + e_1, \cdots, 2^{k-1}z + e_{k-1}\right) \in R_q^k$$

where e is a small error polynomial.

Let $z_0, z_1, \cdots, z_{k-1} \in \mathbb{Z}_q$ be such that the bits of the message $z \in \mathbb{Z}_q$ such that

$$\sum_{i=0}^{n-1} z_i \cdot x^i = z \in R_q$$

Let the elements of $\mathbf{b}$ be denoted by

$$\mathbf{b} = (b_0, b_1, \cdots, b_{k-1}) = \left(z + e_0, 2z + e_1, \cdots, 2^{k-1}z + e_{k-1}\right)$$

The bits of $z_0, z_1, \cdots, z_{k-1}$ can be retrieved in a way similar to Dec_{GSW}. The general decryption algorithm is described below:

initialize $z \leftarrow 0$
for $j = 0, 1, \cdots, n-1$:
$z_j \leftarrow 0$
for $i = k-1, \cdots, 1, 0$:
$z_j \leftarrow z_j + 2^{k-1-i} \cdot \langle b_i - 2^i z_j \notin [-\frac{q}{4}, \frac{q}{4}) \bmod q \rangle \bmod q$
$z \leftarrow z + z_j x^j$

where $\langle x \rangle = \begin{cases} 1 & , x \text{ is true} \\ 0 & , x \text{ is false} \end{cases}$. Output z as the decrypted plaintext.

3.4.6 Homomorphic Operations

Ciphertext Addition

$\mathrm{Add}_{RGSW}(\mathbf{C}_1, \mathbf{C}_2)$: returns the noise-controlled sum of $\mathbf{C}_1$ and $\mathbf{C}_2$.

$$\mathbf{C}_+ = \mathrm{Flatten}_{RGSW}(\mathbf{C}_1 + \mathbf{C}_2) \in R_q^{N \times N}$$

The correctness of this operation is immediate.

Ciphertext Scalar Multiplication

$\mathrm{MultConst}_{RGSW}(\alpha, \mathbf{C})$: returns the noise-controlled scalar product of $\mathbf{C}$ and α.

$$\mathbf{C}_\alpha = \mathrm{Flatten}_{RGSW}(\mathrm{Flatten}_{RGSW}(\alpha \cdot \mathcal{I}_N) \cdot \mathbf{C}) \in R_q^{N \times N}$$

The error increases by a factor of at most N.

The final RGSW scheme is an FHE scheme based on the RLWE problem that overcomes the limitations of previous LWE-based scheme. It achieves polynomial security

in the standard model and has a practical polynomial-time evaluation algorithm. The scheme uses Flattening and Bit Decomposition in a polynomial ring that allows for efficient evaluation. It also achieves significant space complexity reduction, making it more practical for real-world applications.

The major advantage of RGSW over GSW is the significant reduction in size for the private-public key pair and the ciphertext. The size of ciphertext in GSW scheme is $\mathcal{O}\left(n^2 \log^2 q\right)$ whereas it is reduced to just $\mathcal{O}\left(\log^2 q\right)$ in RGSW. This causes great improvements in terms of both execution time and memory.

3.5 Implementation

We have implemented the RGSW scheme in *SageMath*. In this section, we provide and comment on the implementation of RGSW scheme.

The code defines a class called *RingGSW* which contains methods for generating keys, encrypting and decrypting messages, and performing homomorphic operations. The class initializes with a ring, which is a polynomial ring over some finite field. It uses the *sage.all* library for symbolic mathematical operations, including polynomial arithmetic.

The important methods of this class include:

- **keygen** : Generates public and secret keys for encryption and decryption.
- **encrypt** : Encrypts a message using the public key.
- **decrypt** : Decrypts an encrypted message using the secret key.
- **homAdd** : Adds two ciphertexts encrypted with the same public key.
- **homMultConst** : Multiplies the ciphertext with a scalar quantity.

There are also several helper methods that assist in these operations, including **powersOf2**, **bitDecomp**, **bitDecompInv**, **flatten**, **matrixPowersOf2**, **matrixBitDecomp**, **matrixBitDecompInv** and **matrixFlatten**. These methods are used for various arithmetic operations required for encryption and decryption.

```
from sage.all import var, RealDistribution, Integer, Matrix, vector
from math import floor, ceil, log2
from itertools import chain
from random import randrange


class RingGSW:

```

```
9
10      def __init__(self, ring, sigma=5):
11
12          assert ring.variable_name() == 'x'
13
14          self.Rq = ring
15          self.q = self.Rq.base().modulus()
16          self.n = self.Rq.modulus().degree()
17
18          var('x')
19          assert self.Rq.modulus() == self.Rq(x**self.n + 1)
20          assert all([self.q > 1, 1 << floor(log2(self.q)) == self.q])
21          assert all([self.n > 1, 1 << floor(log2(self.n)) == self.n])
22
23          self.k = ceil(log2(self.q))
24          self.N = self.k << 1
25
26          self.gauss = RealDistribution('gaussian', sigma)
27
28
29      def powersOf2(self, pol):
30
31          pol = self.Rq(pol)
32          vec = []
33
34          for i in range(self.k):
35              tpol = self.Rq((1<<i) * pol)
36              vec.append(tpol)
37
38          return vector(self.Rq, vec)
39
40
41      def bitDecomp(self, pol):
42
43          pol = self.Rq(pol)
44          vec = [self.Rq(0)] * self.k
45
46          for i in range(self.n):
47              coeff = Integer(pol[i])
48              for j in range(self.k):
49                  vec[j] += self.Rq((coeff & 1) * x**i)
50                  coeff >>= 1
51
52          return vector(self.Rq, vec)
53
54
55      def bitDecompInv(self, vec):
56
57          assert len(vec) == self.k
58          pol = self.Rq(0)
59
60          for i in range(self.k):
61              pol += (1<<i) * vec[i]
62          pol = self.Rq(pol)
63
64          return pol
65
66
```

```
67      def flatten(self, vec):
68
69          assert len(vec) == self.k
70
71          pol = self.bitDecompInv(vec)
72          vec = self.bitDecomp(pol)
73
74          return vec
75
76
77      def matrixPowersOf2(self, mat):
78
79          nr, nc = mat.nrows(), mat.ncols()
80          newmat = Matrix(self.Rq, nr, [0] * nr * nc * self.k)
81
82          for i in range(nr):
83              v = vector(self.Rq, [])
84              for j in range(nc):
85                  u = self.powersOf2(mat[i][j])
86                  v = vector(self.Rq, chain(v, u))
87              newmat[i, :] = v
88
89          return newmat
90
91
92      def matrixBitDecomp(self, mat):
93
94          nr, nc = mat.nrows(), mat.ncols()
95          newmat = Matrix(self.Rq, nr, [0] * nr * nc * self.k)
96
97          for i in range(nr):
98              v = vector(self.Rq, [])
99              for j in range(nc):
100                 u = self.bitDecomp(mat[i][j])
101                 v = vector(self.Rq, chain(v, u))
102             newmat[i, :] = v
103
104         return newmat
105
106
107     def matrixBitDecompInv(self, mat):
108
109         nr, nc = mat.nrows(), mat.ncols()
110         assert nc % self.k == 0
111
112         newmat = Matrix(self.Rq, nr, [0] * nr * (nc // self.k))
113
114         for i in range(nr):
115             v = vector(self.Rq, [])
116             for j in range(0, nc, self.k):
117                 u = self.bitDecompInv(mat[i, j:j+self.k].row(0))
118                 u = vector(self.Rq, [u])
119                 v = vector(self.Rq, chain(v, u))
120             newmat[i, :] = v
121
122         return newmat
123
124
```

```
125     def matrixFlatten(self, mat):
126
127         assert mat.ncols() % self.k == 0
128
129         newmat = self.matrixBitDecompInv(mat)
130         newmat = self.matrixBitDecomp(newmat)
131
132         return newmat
133
134
135     def keygen(self):
136
137         t = self.Rq.random_element()
138         b1 = self.Rq.random_element()
139
140         err = self.Rq([floor(self.gauss.get_random_element()) for _
    in range(self.n)])
141         b2 = b1 * t + err
142
143         sk = vector(self.Rq, chain(self.powersOf2(1), self.powersOf2
    (-t)))
144         sk = Matrix(self.Rq, self.N, sk)
145         pk = Matrix(self.Rq, 1, [b2, b1])
146
147         return sk, pk
148
149
150     def encrypt(self, pt, pk):
151
152         r = []
153
154         for i in range(self.N):
155             pol = 0
156             for j in range(self.n):
157                 pol += randrange(2) * x**j
158             r.append(pol)
159
160         r = Matrix(self.Rq, self.N, r)
161         ct = self.Rq(pt) * Matrix.identity(self.N)
162         ct += self.matrixBitDecomp(r * pk)
163         ct = self.matrixFlatten(ct)
164
165         return ct
166
167
168     def decrypt(self, ct, sk):
169
170         v = (ct[0:self.k, :] * sk).column(0)
171         pt = 0
172
173         for i in range(self.n):
174             zi = 0
175             for j in range(self.k-1, -1, -1):
176                 b = v[j][i] - (zi<<j)
177                 b = all([b >= self.q>>2, b < 3*(self.q>>2)])
178                 zi += b << (self.k-1-j)
179             pt += zi * x**i
180         pt = self.Rq(pt)
```

```
181
182             return pt
183
184
185         def homMultConst(self, ct, g):
186
187             gct = self.matrixFlatten(self.Rq(g) * Matrix.identity(self.N)
        )
188             gct = self.matrixFlatten(gct * ct)
189
190             return gct
191
192
193         def homAdd(self, ct1, ct2):
194
195             return self.matrixFlatten(ct1 + ct2)
```

3.6 Results

3.6.1 Encryption and Decryption

The results of encryption and decryption time against the modulus q and polynomial degree n are given in the figures below.

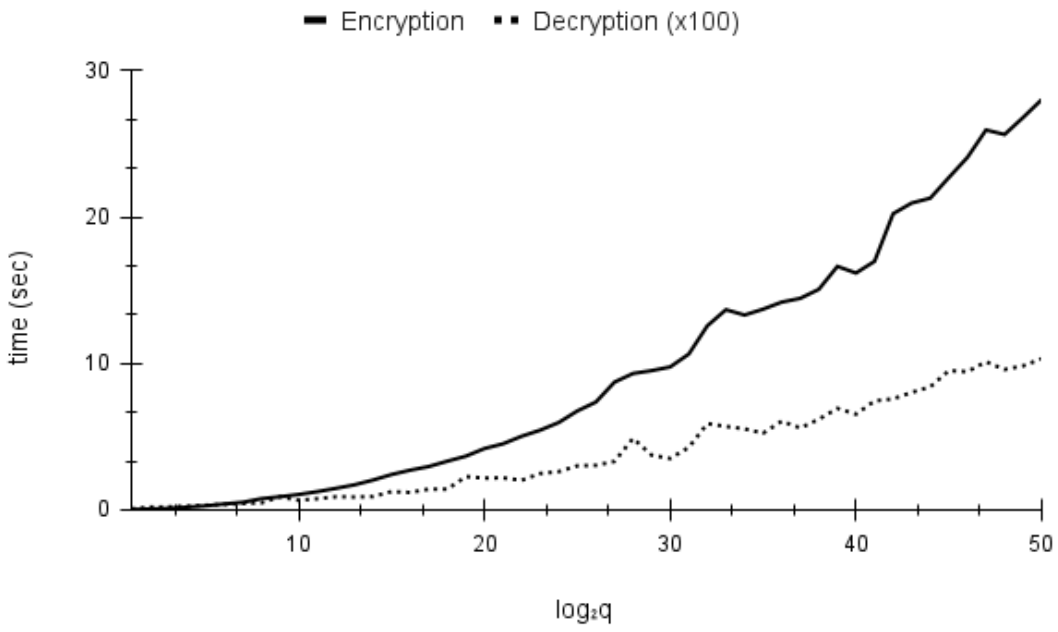


Figure 3.1: Encryption/Decryption time VS $log_2 q$ at $n = 16$

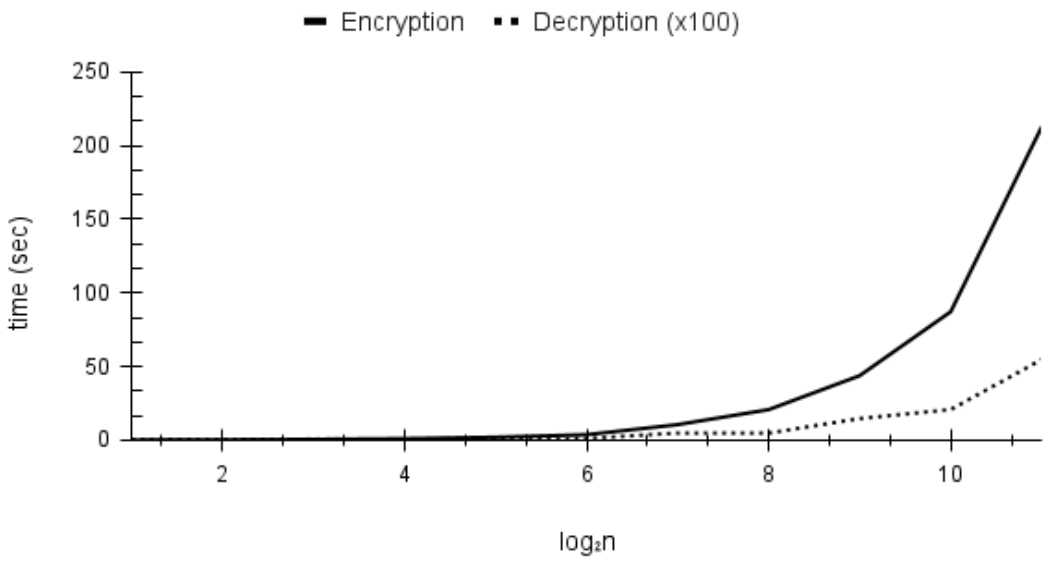


Figure 3.2: Encryption/Decryption time VS $log_2 n$ at $q = 1024$

3.6.2 Homomorphic Operations

The results of execution time for homomorphic operations against the modulus q and polynomial degree n are given in the figures below.

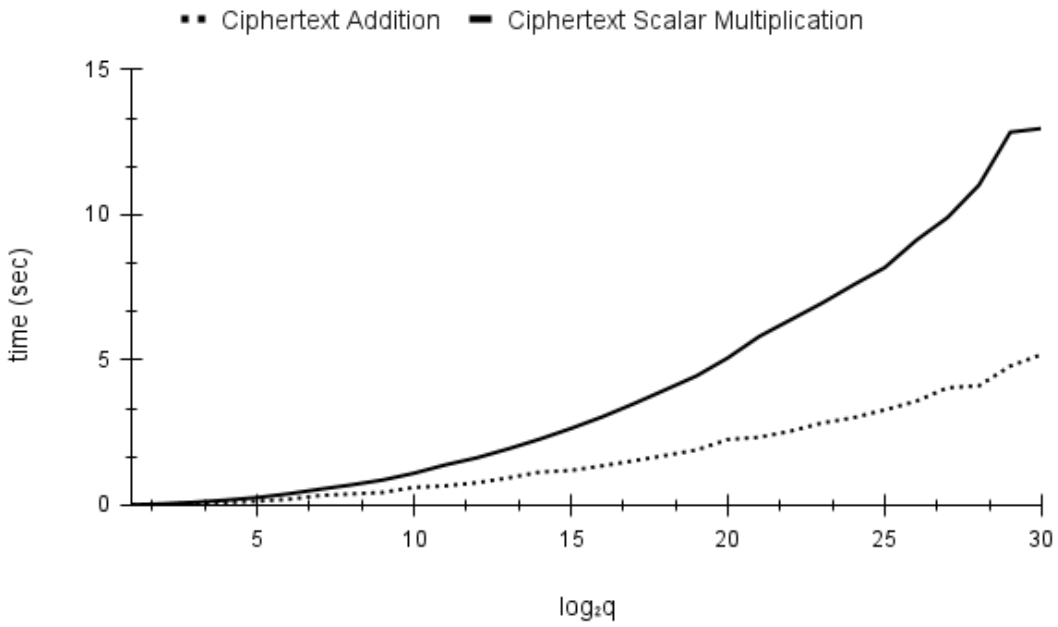


Figure 3.3: Add/MultConst time VS $log_2 q$ at $n = 16$

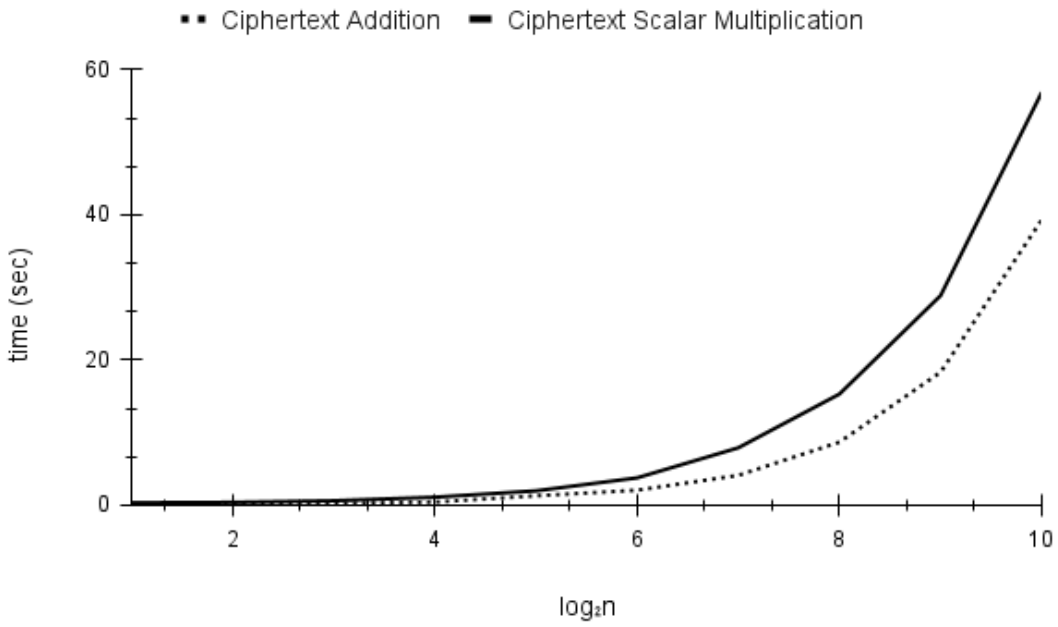


Figure 3.4: Add/MultConst time VS $log_2 n$ at $q = 1024$

Chapter 4

Oblivious Transfer Protocol

4.1 Contributions

We propose a new oblivious transfer protocol based on the RGSW FHE scheme. The proposed protocol is post-quantum secure. Our contributions towards developing this protocol include incorporating the RGSW FHE scheme to enable efficient and secure computations on encrypted data. The proposed protocol utilizes the ring structure of the RGSW scheme to enhance its efficiency and reduce communication costs.

4.2 RGSW based 1-out-of-2 OT Protocol

In this section, we provide the definition and security requirements of OT_1^2. We also propose a post-quantum secure OT_1^2 protocol.

4.2.1 Definition

An OT_1^2 protocol has the following functions:

- Setup: OT parameters are generated and public information is published
- Choose: The receiver selects his/her choice among m_0, m_1, i.e., selects $\sigma \in \{0, 1\}$. Receiver encrypts and sends the encryption c to the sender.
- $\mathrm{KeyDerive}$: The sender computes two keys from c, encrypts the secrets with the keys and sends the encryptions to the receiver.
- $\mathrm{SecretRecover}$: The receiver computes the key w.r.t. σ and decrypts the encryption to get his choice of secret.

4.2.2 Security Requirements

OT_1^2 should meet the following security requirements:

- **Sender's security**: A malicious receiver should only be able to learn about his choice of the secret m_σ and not any other secret $m_i : i \neq \sigma$.
- **Receiver's privacy**: A malicious sender should not be able to learn about the choice(s) σ of the receiver.

4.2.3 Proposed Protocol

Let $\mathcal{S}$ and $\mathcal{R}$ be the sender and receiver respectively. Initial setup is done by invoking Setup_{RGSW}. The sender's private key is sk_S and public key is pk_S. Define a hash function $\mathcal{H} : R_q \to \mathcal{K}$ where $\mathcal{K}$ is the key space of the AES symmetric encryption.

1. The sender $\mathcal{S}$ has secrets m_0, m_1 :
   - Chooses uniformly random $a \in R_q$ and computes $\mathbf{A} = \mathrm{Enc}_{RGSW}(pk_S, a)$
   - Sends $\mathbf{A}$ to the receiver $\mathcal{R}$.
2. The receiver $\mathcal{R}$ makes his choice $\sigma \in \{0, 1\}$:
   - Chooses uniformly random $b \in R_q$
     computes $\mathbf{B} = \begin{cases} \mathrm{Enc}_{RGSW}(pk_S, b) & , \sigma = 0 \\ \mathrm{Add}_{RGSW}(\mathrm{Enc}_{RGSW}(pk_S, b), \mathbf{A}) & , \sigma = 1 \end{cases}$
   - Sends $\mathbf{B}$ to the sender $\mathcal{S}$.
3. The sender $\mathcal{S}$:
   - Computes

$$
\begin{aligned}
d &= \mathrm{Dec}_{RGSW}(sk_S, \mathbf{B}) \in R_q \\
c_0 &= \mathrm{AES}_{Enc}(k_0, m_0) \mid k_0 = \mathcal{H}(d) \in \mathcal{K} \\
c_1 &= \mathrm{AES}_{Enc}(k_1, m_1) \mid k_1 = \mathcal{H}(d - a) \in \mathcal{K}
\end{aligned}
$$

   - Sends c_0, c_1 to the receiver $\mathcal{R}$.
4. The receiver $\mathcal{R}$ computes :

$$
\begin{aligned}
k_\sigma &= \mathcal{H}(b) \in \mathcal{K} \\
m_\sigma &= \mathrm{AES}_{Dec}(k_\sigma, c_\sigma)
\end{aligned}
$$

4.2.4 Flow Diagram

Sender ($\mathcal{S}$)		**Receiver** ($\mathcal{R}$)
Secrets: m_0, m_1		Choice: $\sigma \in \{0, 1\}$
Keys: sk_S, pk_S		Keys: pk_S
$a \in R_q$		
$\mathbf{A} = \mathrm{Enc}_{RGSW}(pk_S, a)$	$\xrightarrow{\mathbf{A}}$	
		$b \in R_q$
		$\mathbf{B} = \begin{cases} \mathrm{Enc}_{RGSW}(pk_S, b) & ,\sigma = 0 \\ \mathrm{Add}_{RGSW}(\mathrm{Enc}_{RGSW}(pk_S, b), \mathbf{A}) & ,\sigma = 1 \end{cases}$
	$\xleftarrow{\mathbf{B}}$	
$d = \mathrm{Dec}_{RGSW}(sk_S, \mathbf{B})$		
$k_0 = \mathcal{H}(d) \in \mathcal{K}$		
$k_1 = \mathcal{H}(d - a) \in \mathcal{K}$		
$c_0 = \mathrm{AES}_{Enc}(k_0, m_0)$		
$c_1 = \mathrm{AES}_{Enc}(k_1, m_1)$		
	$\xrightarrow{c_0, c_1}$	
		$k_\sigma = \mathcal{H}(b) \in \mathcal{K}$
		$m_\sigma = \mathrm{AES}_{Dec}(k_\sigma, c_\sigma)$

4.2.5 Proof of Correctness

It is sufficient to prove that $k_\sigma = k_0$ when $\sigma = 0$ and $k_\sigma = k_1$ when $\sigma = 1$ in order to prove the correctness of the protocol.

If $\sigma = 0$,

$$
\begin{aligned}
k_0 &= \mathcal{H}(d) = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathbf{B})) \\
k_0 &= \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Enc}_{RGSW}(pk_S, b))) \\
k_0 &= \mathcal{H}(b) = k_\sigma
\end{aligned}
$$

If $\sigma = 1$,

$$
\begin{aligned}
k_1 &= \mathcal{H}(d - a) = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathbf{B}) - a) \\
k_1 &= \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Add}_{RGSW}(\mathrm{Enc}_{RGSW}(pk_S, b), \mathbf{A})) - a) \\
k_1 &= \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Enc}_{RGSW}(pk_S, b + a)) - a) \\
k_1 &= \mathcal{H}(b + a - a) = \mathcal{H}(b) = k_\sigma
\end{aligned}
$$

Both sender and receiver compute the same key irrespective of the receiver's choice. Therefore, the receiver would be able to obtain m_σ everytime.

4.3 RGSW based 1-out-of-n OT Protocol

In this section, we provide the definition and security requirements of OT_1^n. We also propose a post-quantum secure OT_1^n protocol. This protocol uses the Add_{RGSW} and MultConst_{RGSW} homomorphic operations to ensure the sender's security as well the receiver's privacy. Like the OT_1^2 protocol, the OT_1^n protocol is a fundamental type of an OT protcol. For example, an OT_k^n protcol can reproduced by running k rounds of an OT_1^n protocol, whereas an OT_1^n cannot be reproduced by the use of an OT_1^2 protocol.

The RGSW FHE scheme allows for computations over ciphertexts represented as a matrix of polynomials with coefficients in a ring. This feature becomes instrumental in improving the computational efficiency. Using the FHE properties, the receiver can perform computations over the ciphertexts without knowledge of their contents. This allows the receiver to compute an OT mask that reveals only the desired message. The receiver then sends this OT mask back to the sender. Finally, the sender can use the OT mask to extract the desired message from all of the ciphertexts.

4.3.1 Definition

An OT_1^n protocol has the following functions:

- Setup: OT parameters are generated and public information is published
- Choose: The receiver selects his/her choice among $m_0, m_1, \cdots, m_{n-1}$, i.e., selects $\sigma \in \{0, 1, \cdots, n-1\}$. Receiver encrypts and sends the encryption c to the sender.
- KeyDerive: The sender computes n keys from c, encrypts the secrets with the keys and sends the encryptions to the receiver.
- SecretRecover: The receiver computes the key w.r.t. σ and decrypts the encryption to get his choice of secret.

4.3.2 Security Requirements

OT_1^n should meet the following security requirements:

- **Sender's security**: A malicious receiver should only be able to learn about his choice of the secret m_σ and not any other secret $m_i : i \neq \sigma$.
- **Receiver's privacy**: A malicious sender should not be able to learn about the choice(s) σ of the receiver.

4.3.3 Proposed Protocol

Let $\mathcal{S}$ and $\mathcal{R}$ be the sender and receiver respectively. Initial setup is done by invoking Setup_{RGSW}. The sender's private key is sk_S and public key is pk_S. Define a hash function $\mathcal{H} : R_q \to \mathcal{K}$ where $\mathcal{K}$ is the key space of the AES symmetric encryption.

1. The sender $\mathcal{S}$ has secrets $m_0, m_1, \cdots, m_{t-1}$:
   - Chooses uniformly random $a \in R_q$ and computes $\mathbf{A} = \text{Enc}_{RGSW}(pk_S, a)$
   - Sends $\mathbf{A}$ to the receiver $\mathcal{R}$.
2. The receiver $\mathcal{R}$ makes his choice $\sigma \in \{0, 1, \cdots, t-1\}$:
   - Chooses uniformly random $b \in R_q$ and computes

$$\mathbf{B} = \text{Enc}_{RGSW}(pk_S, b)$$
$$\mathbf{B} \leftarrow \text{Add}_{RGSW}(\text{MultConst}_{RGSW}(\sigma, \mathbf{A}), \mathbf{B})$$

   - Sends $\mathbf{B}$ to the sender $\mathcal{S}$.
3. The sender $\mathcal{S}$:
   - Computes

$$d = \text{Dec}_{RGSW}(sk_S, \mathbf{B}) \in R_q$$
$$k_i = \mathcal{H}(d - i \cdot a \bmod qR) \in \mathcal{K} \; \forall \; 0 \leq i < t$$
$$c_i = \text{AES}_{Enc}(k_i, m_i) \; \forall \; 0 \leq i < t$$

   - Sends $c_0, c_1, \cdots, c_{t-1}$ to the receiver $\mathcal{R}$.
4. The receiver $\mathcal{R}$ computes :

$$k_\sigma = \mathcal{H}(b) \in \mathcal{K}$$
$$m_\sigma = \text{AES}_{Dec}(k_\sigma, c_\sigma)$$

4.3.4 Flow Diagram

Sender ($\mathcal{S}$)		**Receiver** ($\mathcal{R}$)
Secrets: $m_0, m_1, \cdots, m_{t-1}$		Choice: $\sigma \in \{0, 1, \cdots, t-1\}$
Keys: sk_S, pk_S		Keys: pk_S
$a \in R_q$		
$\mathbf{A} = \mathrm{Enc}_{RGSW}(pk_S, a)$	$\xrightarrow{\mathbf{A}}$	
		$b \in R_q$
		$\mathbf{B} = \mathrm{Enc}_{RGSW}(pk_S, b)$
		$\mathbf{B} \leftarrow \mathrm{Add}_{RGSW}(\mathrm{MultConst}_{RGSW}(\sigma, \mathbf{A}), \mathbf{B})$
	$\xleftarrow{\mathbf{B}}$	
$d = \mathrm{Dec}_{RGSW}(sk_S, \mathbf{B})$		
for $i = 0, 1, \cdots, t-1$:		
$k_i = \mathcal{H}(d - i \cdot a \bmod qR)$		
$c_i = \mathrm{AES}_{Enc}(k_i, m_i)$		
	$\xrightarrow{c_0, c_1, \cdots, c_{t-1}}$	
		$k_\sigma = \mathcal{H}(b) \in \mathcal{K}$
		$m_\sigma = \mathrm{AES}_{Dec}(k_\sigma, c_\sigma)$

4.3.5 Proof of Correctness

Let us begin with deriving a simpler form for k_i w.r.t. the sender $\mathcal{S}$.

$$k_i = \mathcal{H}(d - i \cdot a) = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathbf{B}) - i \cdot a)$$
$$k_i = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Add}_{RGSW}(\mathrm{MultConst}_{RGSW}(\sigma, \mathbf{A}), \mathrm{Enc}_{RGSW}(pk_S, b))) - i \cdot a)$$
$$k_i = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Add}_{RGSW}(\mathrm{Enc}_{RGSW}(pk_S, \sigma \cdot a), \mathrm{Enc}_{RGSW}(pk_S, b))) - i \cdot a)$$
$$k_i = \mathcal{H}(\mathrm{Dec}_{RGSW}(sk_S, \mathrm{Enc}_{RGSW}(pk_S, \sigma \cdot a + b)) - i \cdot a)$$
$$k_i = \mathcal{H}(\sigma \cdot a + b - i \cdot a)$$
$$k_i = \mathcal{H}(b + (\sigma - i) \cdot a)$$

The sender encrypts m_i using k_i as the key. At $i = \sigma$, the receiver's key $k_\sigma = \mathcal{H}(b)$ is the same as computed by the sender. Therefore, the receiver would be able to obtain m_σ everytime.

4.4 Security Model

A security model specfies the security properties that the protocol must satisfy in order to ensure that the exchanged information remains confidential and private. An OT protocol is said to be secure if a malicious receiver is unable to get any information about the sender's secrets (except the receiver's choice). This is known as the sender's security. Additionally, the protocol should also make it infeasible for a malicious sender to gain any information about the receiver's choice. We discuss about a specific security model in this section.

4.4.1 Universal Composability Framework

The Universal Composability (UC) framework is used to analyze the security of cryptographic protocols in a modular and composable way. It was first proposed by R. Canetti in 2001 [5] as a framework to analyze cryptographic protocols that interact with each other in complex ways,

In the UC framework, a cryptographic protocol is modeled as a set of ideal functionalities, which represent abstract operations that the protocol is intended to perform. The protocol is also modeled as a set of non-ideal implementations, which are intended to perform the same operations as the ideal functionalities but may have security vulnerabilities.

For example, in the case of secure multiparty computation (MPC), where multiple parties want to compute a function of their private inputs without revealing any information beyond the function output. The UC framework allows us to model the MPC protocol as a set of ideal functionalities, such as a trusted dealer or a secure function evaluation, and to analyze the security of the protocol by considering its composition with other protocols.

4.5 Implementation

The code implements a OT_1^n protocol using the RGSW cryptosystem. It allows a sender to securely send one out of several messages to a receiver, without knowing which message the receiver has chosen. The sender encrypts a random value a and generates ciphertexts for each message using AES encryption. The receiver homomorphically combines the encrypted $\mathbf{B}$ with the sender's encrypted value $\mathbf{A}$ and uses it to decrypt the ciphertext corresponding to the desired message. The **obliviousTransfer** function performs a single OT operation and returns **True** if successful, otherwise **False**.

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from utils import polynomialHash
from ringgsw import RingGSW


class OTSender:


    def __init__(self, ring, msg_list, sigma=5):

        self.ring = ring
        self.msg_list = msg_list
        self.t = len(self.msg_list)

        self.rgsw = RingGSW(self.ring, sigma)
        self.sk, self.pk = self.rgsw.keygen()

        self.scheme = '1_OUT_OF_N'


    def random_encryption(self):

        self.a = self.ring.random_element()
        A = self.rgsw.encrypt(self.a, self.pk)

        return A


    def generate_ciphertexts(self, B):

        d = self.rgsw.decrypt(B, self.sk)
        ct_list = []

        for i in range(self.t):
            key = polynomialHash(d - (i * self.a))
            aes = AES.new(key, AES.MODE_CBC)
            ct = aes.encrypt(pad(self.msg_list[i], AES.block_size))
            ct_list.append(aes.iv + ct)

        return ct_list


class OTReceiver:


    def __init__(self, ring, pk, e, t):

        assert all([e >= 0, e < t])

        self.ring = ring
        self.rgsw = RingGSW(self.ring)
        self.pk = pk

        self.e = e
        self.t = t

        self.scheme = '1_OUT_OF_N'
```

```
59
60
61      def homomorphic_encryption(self, A):
62
63          self.b = self.ring.random_element()
64          B = self.rgsw.encrypt(self.b, self.pk)
65
66          e_A = self.rgsw.homMultConst(A, self.e)
67          B = self.rgsw.homAdd(e_A, B)
68
69          return B
70
71
72      def decrypt(self, ct_list):
73
74          key = polynomialHash(self.b)
75          ct = ct_list[self.e]
76          iv, ct = ct[:AES.block_size], ct[AES.block_size:]
77
78          aes = AES.new(key, AES.MODE_CBC, iv)
79          m_e = unpad(aes.decrypt(ct), AES.block_size)
80
81          return m_e
82
83
84  def obliviousTransfer(ring, msg_list, e, sigma=5):
85
86      sender = OTSender(ring, msg_list, sigma)
87      client = OTReceiver(ring, sender.pk, e, sender.t)
88
89      A = sender.random_encryption()
90      B = client.homomorphic_encryption(A)
91      ct_list = sender.generate_ciphertexts(B)
92      m_e = client.decrypt(ct_list)
93
94      return msg_list[e] == m_e
```

4.6 Results

The results of execution time against the modulus q, polynomial degree n and the number of secrets N for OT_1^N are shared in this section.

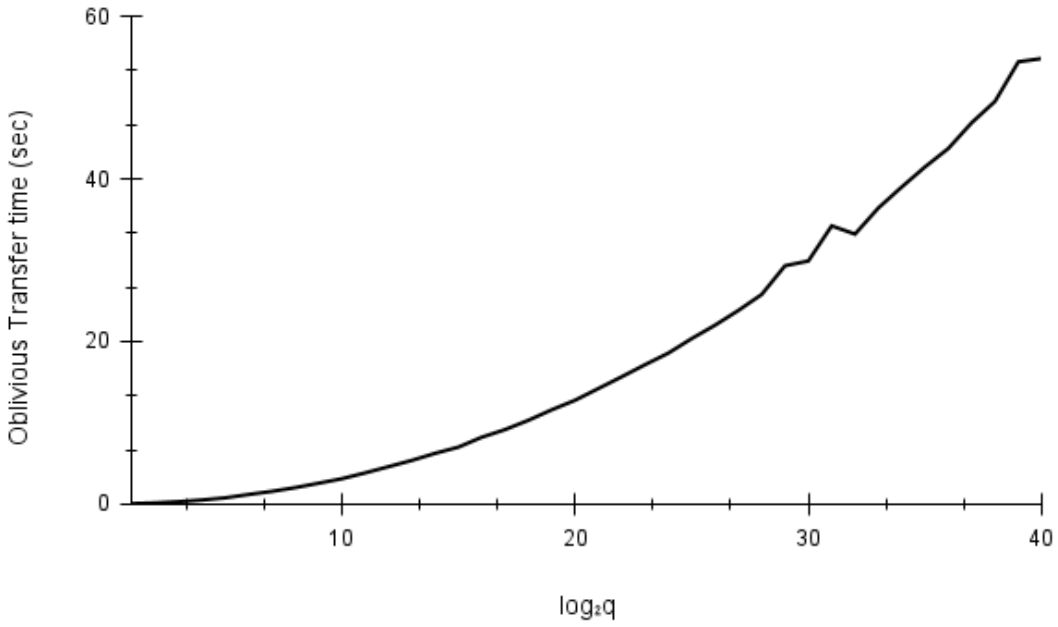


Figure 4.1: Oblivious transfer time VS $log_2 q$ at $n = 16, N = 20$

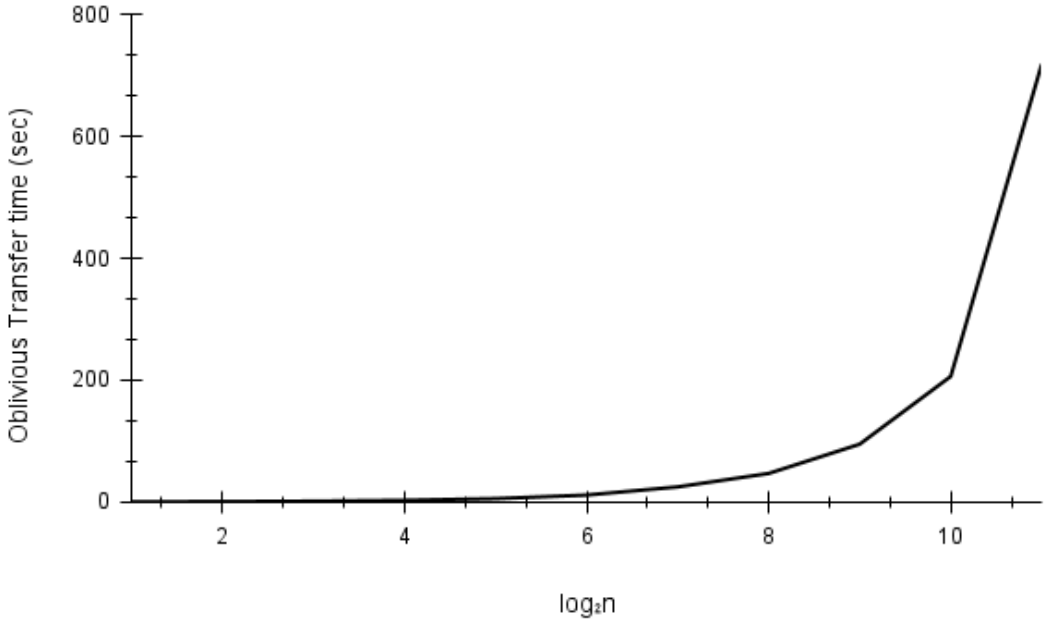


Figure 4.2: Oblivious transfer time VS $log_2 n$ at $q = 1024, N = 20$

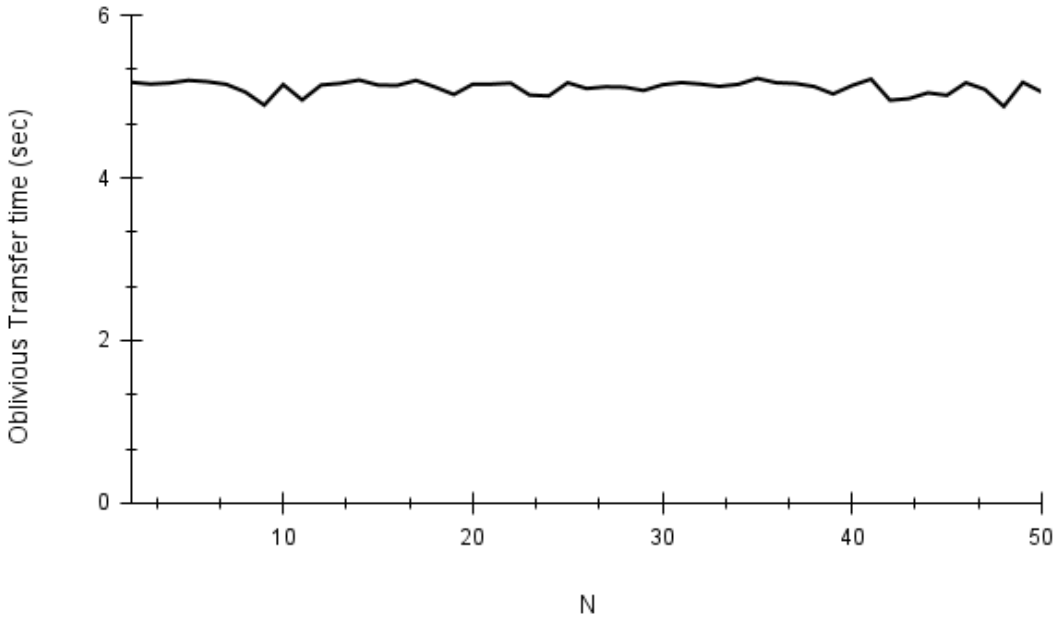


Figure 4.3: Oblivious transfer time VS N at $n = 16, q = 8192$

Chapter 5

Conclusion

5.1 Work Done

We have succesfully proposed, constructed, implemented and tested the following:

1. The RGSW FHE scheme which has homomorphic properties and provides post-quantum security.
2. 1-out-of-2 and 1-out-of-n oblivious transfer protocols based on the RGSW FHE scheme which is secure under the RLWE assumption.

We have successfully proven the correctness of the RGSW FHE scheme and the oblivious transfer protocols both theoretically and practically.

5.2 Future Scope

We have shown OT_1^2 and OT_1^n. Further study in this area shows promise for the development of an efficient RGSW based k-out-of-n oblivious transfer protocol, i.e., OT_k^n.

A practical OT_k^n protocol would enable the sharing of multiple secrets in a single communication round, which would result in a significant reduction in communication overhead. It would also bring a significant improvement in the time complexity of sharing k secrets simultaneously. It can also open up new areas of research in the field of private information retrieval.

Bibliography

[1] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, page 84–93, New York, NY, USA, 2005. Association for Computing Machinery.

[2] Chris Peikert and Zachary Pepin. Algebraically structured lwe, revisited. *IACR Cryptology ePrint Archive*, pages 1–23, 11 2019.

[3] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *Advances in Cryptology – CRYPTO 2013*, pages 75–92, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[4] Saeid Esmaeilzade, Nasrollah Pakniat, and Ziba Eslami. A generic construction to build simple oblivious transfer protocols from homomorphic encryption schemes. *The Journal of Supercomputing*, 78, 01 2022.

[5] R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. *IACR Cryptology ePrint Archive*, 2000:136 – 145, 11 2001.

[6] Joe Kilian. Founding crytpography on oblivious transfer. In *Proceedings of the Twentieth Annual ACM Symposium on Theory of Computing*, STOC '88, page 20–31, New York, NY, USA, 1988. Association for Computing Machinery.

[7] A. K. Lenstra, H. W. Lenstra, and L. Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, December 1982.

[8] Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, pages 700–718, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[9] Léo Ducas and Daniele Micciancio. Fhew: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015*, pages 617–640, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[10] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology – ASIACRYPT 2016*, pages 3–33, Berlin, Heidelberg, 2016. Springer Berlin Heidelberg.

[11] Christian Mouchet, Juan Troncoso-Pastoriza, Jean-Philippe Bossuat, and Jean-Pierre Hubaux. Multiparty homomorphic encryption from ring-learning-with-errors. *Proceedings on Privacy Enhancing Technologies*, 2021:291–311, 10 2021.

[12] Xiaojun Zhang, Chunxiang Xu, Chunhua Jin, Run Xie, and Jining Zhao. Efficient fully homomorphic encryption from rlwe with an extension to a threshold encryption scheme. *Future Generation Computer Systems*, 36, 01 2013.

[13] Yu Zhi-Min, Jing Zheng-Jun, and Li Shi-Cun. Diffie-hellman key exchange protocol based on ring-lwe. *The Open Cybernetics & Systemics Journal*, 9:1033–1037, 09 2015.

[14] Saikrishna Badrinarayanan, Daniel Masny, and Pratyay Mukherjee. Efficient and tight oblivious transfer from pke with tight multi-user security. In Giuseppe Ateniese and Daniele Venturi, editors, *Applied Cryptography and Network Security*, pages 626–642, Cham, 2022. Springer International Publishing.

[15] Zengpeng Li, Can Xiang, and Chengyu Wang. Oblivious transfer via lossy encryption from lattice-based cryptography. *Wireless Communications and Mobile Computing*, 2018:1–11, 09 2018.

Appendices

Appendix A

Implementation of RGSW and OT protocol

A.1 ring_gsw.py

The code in section 3.5 is the implementation of the RGSW cryptosystem in SageMath. The **RingGSW** class provides abstract methods for key generation, encryption, decryption and homomorphic operations.

A.2 ot_1_n.py

The code in section 4.5 is the implementation of our OT_1^n protocol. The **OTSender** and **OTReceiver** classes instantiate a sender and receiver respectively. The **obliviousTransfer** method simualtes a 1-out-of-n oblivious transfer of random secrets between the sender and receiver.

Appendix B

Simulation and Testing

B.1 server.py

The code implements a basic protocol for oblivious transfer (OT) between a sender and a receiver over a network connection. The sender generates a set of random messages, creates an OT sender object, and shares the parameters of the ring over the network. The protocol involves the exchange of encrypted messages and symmetric key encryption using AES encryption. The code uses the SageMath library and several helper functions to facilitate the OT protocol.

```
from sage.all import var, PolynomialRing, Zmod
from utils import server_socket, send_data, recv_data
import ot_1_N
from os import urandom
import pickle


def on_new_connect(client, msg_list):

    q, n, sigma = 2**17, 2**4, 5
    var('x')

    ring = PolynomialRing(Zmod(q), 'x').quotient(x**n + 1, 'x')
    send_data(client, pickle.dumps(ring))

    ot_sender = ot_1_N.OTSender(ring, msg_list, sigma)
    send_data(client, pickle.dumps(ot_sender.pk))
    send_data(client, pickle.dumps(len(msg_list)))
    recv_data(client)

    return ot_sender


def ot_1_out_of_N(client, ot_sender):

    print('\nsending a random encryption (A)\n')
    A = ot_sender.random_encryption()
    send_data(client, pickle.dumps(A))

```

```
30      B = pickle.loads(recv_data(client))
31      print('recieved homomorphic encryption of A (B)\n')
32
33      print('sending symmetric AES ciphertexts\n')
34      ct_list = ot_sender.generate_ciphertexts(B)
35      send_data(client, pickle.dumps(ct_list))
36
37
38  def main(host, port):
39
40      server_sock = server_socket(host, port)
41      print(f'\n[*] listening on port {port}\n')
42
43      client, _ = server_sock.accept()
44      print('connection accepted\n')
45      msg_list = [urandom(16) for _ in range(6)]
46
47      print('messages (visible to sender):\n')
48      for i in range(len(msg_list)):
49          print(f'{i}. {msg_list[i].hex()}')
50
51      print('\nsharing parameters\n')
52      ot_sender = on_new_connect(client, msg_list)
53
54      ot_1_out_of_N(client, ot_sender)
55
56      server_sock.close()
57      print('[*] connection closed\n')
58
59
60  if __name__ == '__main__':
61
62      main('localhost', 3000)
```

B.2 client.py

This is a Python script for a client program that connects to a server program using sockets. The program implements oblivious transfer (OT) protocols, where the client and server exchange encrypted messages without revealing any information about them to each other. When the client connects to the server, it receives parameters including a ring object and a public key. The client receives an encrypted message from the server, performs some operations on it using the OT protocol, and then decrypts the message. The decrypted message is printed to the console. Finally, the connection to the server is closed.

```
from utils import send_data, recv_data
import ot_1_N
import socket
import pickle


def on_connect(s):

    ring = pickle.loads(recv_data(s))
    sender_pk = pickle.loads(recv_data(s))
    t = pickle.loads(recv_data(s))

    choice = int(input(f'\nenter your choice (0 to {t-1}): '))
    ot_recv = ot_1_N.OTReceiver(ring, sender_pk, choice[0], t)

    send_data(s, pickle.dumps(ot_recv.scheme))

    return ot_recv, choice


def ot_1_out_of_N(s, ot_recv):

    A = pickle.loads(recv_data(s))
    print('\nrecevied the random encryption (A)\n')

    print('sending homomorphic encryption of A (B)\n')
    B = ot_recv.homomorphic_encryption(A)
    send_data(s, pickle.dumps(B))

    ct_list = pickle.loads(recv_data(s))
    print('recevied symmetric AES ciphertexts:\n')
    for i in range(len(ct_list)):
        print(f'{i}. {ct_list[i].hex()}')

    return ot_recv.decrypt(ct_list)


def main(host, port):

    s = socket.socket()
    s.connect((host, port))
    print('\nconnected to sender\n')
```

```
43
44      try:
45
46          print('received parameters (ring, public key)')
47          ot_recv, choice = on_connect(s)
48
49          msg = ot_1_out_of_N(s, ot_recv)
50          print(f'\ndecrypted message:\n{choice[0]}. {msg.hex()}\n')
51
52      except Exception as e:
53          print(str(e))
54
55      s.close()
56      print('\n[*] connection closed\n')
57
58
59  if __name__ == '__main__':
60
61      main('localhost', 3000)
```

B.3 utils.py

The given code contains utility functions for sending and receiving data over sockets and for computing the hash of a polynomial. These functions can be used to facilitate socket communication and polynomial hashing.

```
from sage.all import PolynomialRing, ZZ
import socket
from Crypto.Util.number import long_to_bytes, bytes_to_long
from hashlib import sha256


HEADER_SIZE = 10


def server_socket(host, port):

    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    s.bind((host, port))
    s.listen()
    return s


def send_data(c, data: bytes):

    np = (len(data) >> 12) + ((len(data) & (1<<12)-1) > 0)
    np_bytes = long_to_bytes(np)
    header = f'{hex(len(np_bytes))[2:]:>0{HEADER_SIZE}}'.encode()
    c.send(header + np_bytes)

    i = 0
    while i < np:
        j = i<<12
        if i < np-1:
            c.send(data[j: j+(1<<12)])
        else:
            data = data[j:]
            len_bytes = long_to_bytes(len(data))
            header = f'{hex(len(len_bytes))[2:]:>0{HEADER_SIZE}}'.
    encode()
            c.send(header + len_bytes)
            data = data + b'\x00'*((1<<12) - len(data))
            c.send(data)
        i += 1


def recv_data(c):

    header = c.recv(HEADER_SIZE)
    data = c.recv(int(header.decode(), 16))
    np = bytes_to_long(data)

    data = b''
    i = 0
```

```
49      while i < np:
50          if i < np-1:
51              data += c.recv(1<<12)
52          else:
53              len_bytes = c.recv(HEADER_SIZE)
54              len_bytes = c.recv(int(len_bytes.decode(), 16))
55              data += c.recv(1<<12)[:bytes_to_long(len_bytes)]
56          i += 1
57
58      return data
59
60
61  def polynomialHash(pol):
62
63      R = PolynomialRing(ZZ, 'x')
64      pol = R(list(pol))
65
66      h = long_to_bytes(pol.subs(x=2))
67      h = sha256(h).digest()
68
69      return h
```

B.4 test.py

This is a Python code implementing tests for a cryptographic scheme. The code includes a test class **RingGSWOTTests** with several methods to test the implemented cryptographic functionalities. The **RingGSWOTTests** class tests the correctness of several cryptographic operations including **bitDecomp**, **flatten**, **encrypt**, **decrypt**, **homAdd**, **homMultConst**, and the RGSW based OT protocol.

```
from sage.all import var, PolynomialRing, Zmod
from math import floor, ceil, log2
from random import randrange
from ringgsw import RingGSW
import ot_1_N


class RingGSWOTTests:


    def __init__(self, q, n, sigma=5):

        assert all([q > 1, 1 << floor(log2(q)) == q])
        assert all([n > 1, 1 << floor(log2(n)) == n])

        self.k = ceil(log2(q))
        self.sigma = sigma

        var('x')
        self.Zq = Zmod(q)
        F = PolynomialRing(self.Zq, 'x')
        self.Rq = F.quotient(x**n + 1, 'x')

        self.rgsw = RingGSW(self.Rq, self.sigma)
        self.sk, self.pk = self.rgsw.keygen()


    def dotProduct(self, v1, v2):

        assert len(v1) == len(v2)

        prod = self.Rq(0)
        for u, v in zip(v1, v2):
            prod += u * v

        return prod


    def propertiesTest(self):

        a = self.Rq.random_element()
        b = self.Rq.random_element()
        v = [self.Rq.random_element() for _ in range(self.k)]

        try:
```

```
46              assert self.rgsw.bitDecompInv(self.rgsw.bitDecomp(a)) ==
    a
47
48              w = self.rgsw.flatten(v)
49              t = self.rgsw.bitDecomp(a)
50              y = self.rgsw.powersOf2(b)
51              z = self.rgsw.bitDecompInv(v)
52
53              assert all([
54                  self.dotProduct(t, y) == a * b,
55                  self.dotProduct(v, y) == self.dotProduct(w, y) == z *
     b
56              ])
57
58              print("powersOf2, bitDecomp, bitDecompInv, flatten [OK]\n
    ")
59
60          except Exception as e:
61              print("powersOf2, bitDecomp, bitDecompInv, flatten [
    Failed]")
62              print(f"\n{str(e)}\n")
63
64
65      def encryptDecryptTest(self):
66
67          m = self.Rq.random_element()
68          ct = self.rgsw.encrypt(m, self.pk)
69          z = self.rgsw.decrypt(ct, self.sk)
70
71          try:
72              assert z == m
73              print("encryption, decryption [OK]\n")
74
75          except AssertionError:
76              print("encryption, decryption [Failed]")
77              print(f"\nactual: {m}\n\nobtained: {z}\n")
78
79
80      def homMultConstTest(self, L=20):
81
82          msg = self.Rq.random_element()
83          gct = self.rgsw.encrypt(msg, self.pk)
84
85          i = 0
86
87          try:
88              while i < L:
89
90                  g = self.Zq.random_element()
91                  if g == 0: continue
92
93                  gct = self.rgsw.homMultConst(msg, g)
94                  msg *= self.Rq(g)
95
96                  z = self.rgsw.decrypt(gct, self.sk)
97                  assert z == msg
98
99                  i += 1
```

```
100
101             print("homomorphic constant multiplication [OK]\n")
102
103         except AssertionError:
104             print(f"homomorphic constant multiplication [Failed at op
    . {i+1}/{L}]")
105             print(f"\nactual: {msg}\n\nobtained: {z}\n")
106
107
108     def homAddTest(self, L=20):
109
110         msg = self.Rq.random_element()
111         hct = self.rgsw.encrypt(msg, self.pk)
112
113         i = 0
114
115         try:
116             while i < L:
117
118                 m = self.Rq.random_element()
119                 ct = self.rgsw.encrypt(m, self.pk)
120
121                 hct = self.rgsw.homAdd(hct, ct)
122                 msg += m
123
124                 z = self.rgsw.decrypt(hct, self.sk)
125                 assert z == msg
126
127                 i += 1
128
129             print("homomorphic addition [OK]\n")
130
131         except AssertionError:
132             print(f"homomorphic addition [Failed at op. {i+1}/{L}]")
133             print(f"\nactual: {msg}\n\nobtained: {z}\n")
134
135
136     def ot_1_out_of_N_Test(self, N=20):
137
138         e = randrange(N)
139         messages = [bytes([randrange(256) for _ in range(10)]) for _
    in range(N)]
140
141         try:
142             assert ot_1_N.obliviousTransfer(self.Rq, messages, e,
    self.sigma)
143             print("1 out of N oblivious transfer [OK]\n")
144
145         except Exception as e:
146             print(f"1 out of N oblivious transfer [Failed]")
147             print(f"\n{str(e)}\n")
148
149
150     def runAllTests(self):
151
152         self.propertiesTest()
153         self.encryptDecryptTest()
154         self.homMultConstTest()
```

```
155         self.homAddTest()
156         self.ot_1_out_of_N_Test()
157 
158 
159 def main():
160 
161     test = RingGSWOTTests(q=2**17, n=2**4, sigma=5)
162     test.runAllTests()
163 
164 
165 if __name__ == '__main__':
166 
167     main()
```